

ГОТОВНОСТЬ

8.0¹⁰

Production-ready (pending QA)

Телеметрия поездов

Production-grade спецификация: **изоляция Worker-процесса, snap-to-grid кэш, STRIDE threat model, 3-уровневый backup, ADMIN_TOKEN, AlertManager.**

Сервис сбора, хранения и анализа GPS-данных

автомобильных поездов с план-фактным анализом.

Архитектура: Next.js 16 + Prisma + Turso/LibSQL, Worker-

процесс для асинхронной обработки TrafficJob, Redis sliding

window rate-limit, circuit breaker для 2ГИС API. Безопасность:

STRIDE threat model (18 угроз), идемпотентность через clientId,

payload-лимиты. Данные: retention 10 лет, soft-delete с grace

period, Backup Job (type=full) с верификацией checksum.

Наблюдаемость: Prometheus-метрики, машинночитаемые

правила AlertManager (6 alerts). Масштабирование: явные

потолки SQLite single-writer, порог миграции на PostgreSQL.

АРХИТЕКТУРА	БЕЗОПАСНОСТЬ	ПРОИЗВОДИТЕЛЬНОСТЬ	НАБЛЮДАЕМОСТЬ	ТЕСТИРОВАНИЕ	DEVOPS / CI
8.5	9.0	8.5	8.5	7.0	8.5

КЛЮЧЕВЫЕ АРХИТЕКТУРНЫЕ РЕШЕНИЯ

- W** Worker изолирован от API
- I** Идемпотентность clientId
- C** Кэш snap-to-grid ~55м
- T** Threat model STRIDE (18)
- B** Backup 3 уровня, RPO 1ч
- R** Rate limit Redis sliding
- A** ADMIN_TOKEN для admin API
- D** Деградация circuit breaker
- M** Миграции expand-contract
- S** Scale limits + порог PG

Production-grade спецификация

Полный набор production-гарантий: безопасность, надёжность, сохранность данных, наблюдаемость. Незавершённые работы: реальные k6-замеры и покрытие тестами (симуляция в таблицах 12, 11.1).

Версия: 2.6

Дата:

29.08.2026

Статус:

Production-

ready

(pending QA)

Содержание

1. Обзор системы	4
1.1 Назначение	4
1.2 Ключевые возможности	4
1.3 Стек технологий	4
1.4 Производственные гарантии	5
2. Архитектура	7
2.1 Общая схема	7
2.2 Структура проекта	7
2.3 Схема данных (Prisma)	8
2.4 Middleware	11
2.5 Контракты модулей	12
2.6 Изоляция Worker-процесса	13
3. Принципы работы	16
3.1 Сбор GPS-данных (правки #5, #6)	16
3.2 Цепочка маршрутизации	16
3.3 Исторические пробки (Traffic) — обработка в Worker	16
3.4 План-фактный анализ	17
3.5 Кэширование маршрутов	17
3.6 Идемпотентность ingest	19
3.7 Политика хранения данных — 10 лет	20
3.8 Политика деградации	21
3.9 Ограничения масштабирования	22
4. API-эндпоинты	24
4.1 POST /api/ingest (правки #5, #6)	24
4.2 GET /api/sessions — фильтры	24
4.3 GET /api/sessions/[id]	25
4.4 POST /api/plan	25
4.5 GET /api/plan/[sessionId]	25
4.6 CRUD /api/routes	25
4.7 Валидация входных данных	26
4.8 GET /health	26

4.9 Worker API — внутренние endpoints	26
4.10 DELETE /api/sessions/[id]	28
4.11 POST /api/sessions/[id]/export	28
4.12 GET /api/exports/[jobId] — poll	30
5. Фронтенд	31
5.1 Архитектура компонентов	31
5.2 Карта (MapContainer)	31
5.3 Визуальная тема	31
6. Безопасность	32
6.1 Авторизация	32
6.2 Валидация входных данных	32
6.3 Rate Limiting	32
6.4 CORS	34
6.5 Security Headers	34
6.6 Управление секретами	34
6.7 Идемпотентность как механизм защиты	34
6.8 Audit Log и compliance	35
6.9 Threat Model	35
7. Наблюдаемость	38
7.1 Логирование	38
7.2 Метрики	38
7.3 Health-check и алертинг	39
8. Тестирование	41
8.1 Unit-тесты	41
8.2 Integration-тесты	42
8.3 E2E-тесты	42
8.4 Нагрузочные тесты	42
8.5 Контрактные тесты	43
9. Развёртывание	44
9.1 Локальная разработка	44
9.2 CI/CD-конвейер	44
9.3 Миграции базы данных — Backward-Compatible	44
9.4 Продакшн (Turso + Vercel/Docker)	46

9.5 Среда staging и rollback	46
9.6 Развёртывание Worker-процесса	46
9.7 Сгон-задачи и poll-циклы в Worker	47
9.8 Резервное копирование	48
10. Зависимости	51
11. Переменные окружения	52

1. Обзор системы

1.1 Назначение

Сервис «Телеметрия поездок» предназначен для непрерывного сбора, хранения и анализа GPS-данных автомобильных поездок. Основная ценность системы заключается в план-фактном анализе: сервис сравнивает фактически пройденный маршрут с заранее запланированным, вычисляя отклонения по каждому сегменту пути, длительности, скорости и пробковым условиям. Это позволяет оценивать эффективность маршрутизации, качество дорожной инфраструктуры и поведение водителя в реальных условиях.

Система ориентирована на личное использование: владелец устанавливает приложение Sensor Logger на iPhone, которое записывает GPS-координаты с высокой частотой. Данные автоматически отправляются на сервер через API-эндпоинт ingest. Веб-интерфейс позволяет просматривать поездки на карте, анализировать скорость, выявлять проблемные участки и сравнивать план с фактом. Все данные хранятся в базе SQLite (Turso/LibSQL), что обеспечивает простоту развёртывания и минимальные операционные затраты.

1.2 Ключевые возможности

- Сбор GPS-треков из Sensor Logger (iPhone) через HTTP API с Bearer-авторизацией и **идемпотентной передачей** (clientId).
- Автоматическая маршрутизация через API 2ГИС (carrouting 6.0.0) с fallback на OSRM и гаверсинус.
- Исторические данные о пробках по каждому сегменту маршрута — обработка выделена в **отдельный Worker-процесс**.
- План-фактный анализ: отклонение по длительности, скорости и дистанции для каждого сегмента.
- Веб-интерфейс с декомпозированными компонентами, интерактивной картой (Leaflet) и 4 вкладками телеметрии.
- Управление избранными маршрутами (сохранение, удаление, использование при планировании).
- Двухуровневое кэширование результатов маршрутизации (in-memory LRU + SQLite persistent) с **округлением до сетки ~55 м × ~35 м (snap-to-grid)** и time-of-day-ключом.
- Cursor-based пагинация GPS-точек для эффективной загрузки больших сессий.
- Структурированное логирование (pino), Prometheus-метрики и health-check.
- Zod-валидация всех входных данных, **payload-лимиты (256 КБ)**, **p-limit concurrency control**, CORS и security headers.
- **Backward-compatible миграции** БД по паттерну expand-contract; CI/CD-конвейер с автоматическими тестами.
- Rate limiting на **Redis sliding window** (multi-instance) с fallback на in-memory для single-instance.
- **STRIDE threat model** (18 угроз), **политика деградации** 2ГИС→OSRM→гаверсинус с circuit breaker и уведомлением пользователя, **3-уровневый backup** (Turso snapshots + logical dump + on-demand export) с RPO 1ч / RTO 30мин.

1.3 Стек технологий

Слой	Технология	Назначение
Фронтенд	Next.js 16 (App Router)	SSR/SSG, React Server Components, API Routes
Стилизация	Tailwind CSS 4 + oklch	Адаптивная вёрстка, CSS-переменные темы
Карта	React-Leaflet + Leaflet	Интерактивная карта с GPS-треком и маркерами
Валидация	Zod	Схемы валидации API-запросов
ORM	Prisma	Типобезопасные запросы к БД с индексами
БД (prod)	Turso / LibSQL	Реплицируемая SQLite
БД (dev)	SQLite (локальный)	Локальная разработка
Маршрутизация	2ГИС carrouting 6.0.0	Построение маршрутов и получение пробок
Fallback	OSRM Demo Server	Резервная маршрутизация
Fallback	Гаверсинус (40 км/ч)	Расчёт прямой дистанции
Логирование	Pino	JSON-логи с уровнями и контекстом
Метрики	prom-client	Prometheus-метрики (/metrics)
Очередь задач	Таблица TrafficJob + Worker Process	Изоляция асинхронной обработки от API event loop
Rate limit store	Redis (Upstash) / in-memory LRU	Sliding window для multi-instance
Circuit breaker	opossum	защита от лавины timeout-ов при отказе 2ГИС
Backup (managed)	Turso automated snapshots	point-in-time recovery, ежедневно
Backup (logical)	ExportJob JSON + scripts/restore-backup.ts	ежедневный dump, 90 дней rolling
Тестирование	Vitest + Playwright + k6	Unit, integration, E2E, нагрузочные тесты
Деплой	Docker / Vercel Pro	Контейнеризация (рекомендуется) или Vercel Pro (Cron 1 мин); free-tier BLOCKING для 100 сессий/мин

Таблица 1. Технологический стек сервиса.

1.4 Производственные гарантии

Система спроектирована с учётом production-grade требований. Ключевые гарантии: (а) **безопасность** — STRIDE threat model (см. §6.9), Zod-валидация всех входов, payload-лимиты, rate limiting на Redis sliding window, разделение токенов по scope; (б) **надёжность** — Worker-процесс изолирован от API event loop, верифицируемый захват задач через UPDATE ... RETURNING id, circuit breaker для 2ГИС, политика деградации с уведомлением пользователя (см. §3.8); (в) **сохранность данных** — retention 10 лет (см. §3.7), 3-уровневый backup с RPO 1ч / RTO 30мин (см. §9.8), soft-delete с grace period 30 дней, AuditLog всех destructive-операций; (г) **наблюдаемость** — Prometheus-метрики, машиночитаемые правила AlertManager (см. §7.3), структурированное логирование Pino с requestId; (д) **масштабируемость** — горизонтальное масштабирование Worker и API (stateless), явные потолки и порог миграции на PostgreSQL (см. §3.9); (е) **миграции БД** — backward-compatible (expand-contract), CI-проверка

деструктивных операций (см. §9.3).

2. Архитектура

2.1 Общая схема

Архитектура сервиса построена по клиент-серверной модели с **четырьмя** основными компонентами: мобильный клиент (Sensor Logger на iPhone), API-сервер (Next.js API Routes + Prisma + LibSQL), **Worker-процесс** (отдельный Node.js-процесс для обработки TrafficJob) и веб-клиент (React-фронтенд с Leaflet-картой). Взаимодействие защищено Bearer-токенами и проходит через единый middleware-слой, обеспечивающий авторизацию, rate limiting, CORS, security headers и логирование.

Все GPS-данные поступают через endpoint ingest, где проходят Zod-валидацию, проверку payload size (≤ 256 КБ), нормализацию и сохранение в БД. Создание TrafficJob теперь является **единственной неблокирующей операцией** в ingest-хендлере: INSERT в таблицу jobs со статусом pending. Вся тяжёлая асинхронная работа (traffic-fetch к 2ГИС, retry, обогащение сегментов) выполняется в Worker-процессе, что устраняет блокировку event loop API-сервера при пиковой нагрузке (100 сессий/мин).

2.2 Структура проекта

Проект организован по модульному принципу с чётким разделением ответственности. Директория worker/ содержит Worker-процесс, src/lib/idempotency.ts — логику идемпотентности.

Путь	Назначение
src/app/	Next.js App Router: страницы и API routes
src/middleware.ts	Сквозная обработка: async, авторизация, rate-limit, CORS, requestId, payload-size guard, try/catch
src/lib/routing.ts	Цепочка маршрутизации (2ГИС → OSRM → гаверсинус) с circuit breaker
src/lib/circuit-breaker.ts	circuit breaker для 2ГИС API (opossum)
src/lib/traffic.ts	Параллельный fetch данных о пробках с controlled concurrency (вызывается из Worker)
src/lib/plan.ts	План-фактный анализ: сегментация, отклонения
src/lib/validation.ts	Zod-схемы для всех API endpoints (включая clientId)
src/lib/cache.ts	Двухуровневый кэш маршрутизации (LRU + SQLite), snap-to-grid ~55 м + ToD
src/lib/idempotency.ts	проверка (deviceId, clientId), возврат существующего sessionId
src/lib/rate-limit.ts	Redis sliding window + in-memory fallback
src/lib/export.ts	генерация GPX 1.1 / KML 2.2 / JSON (xmlbuilder2)
src/lib/retention.ts	выбор кандидатов, авто-архивация, hard delete
src/lib/backup.ts	логический дамп БД (BackupJob), верификация целостности
src/lib/logger.ts	Pino-логгер с requestId и контекстом
src/lib/metrics.ts	Prometheus-метрики (prom-client)

Путь	Назначение
worker/index.ts	точка входа Worker-процесса (poll TrafficJob, fetch traffic)
worker/processor.ts	обработка одной TrafficJob с retry и backoff (RETURNING-захват)
worker/backup-runner.ts	выполнение BackupJob, верификация, retry
src/components/	Декомпозированные React-компоненты
prisma/	Схема БД и миграции (expand-contract, проверка check-migrations.sh)
scripts/check-migrations.sh	реальный lint деструктивных миграций (prisma migrate diff + grep)
scripts/restore-backup.ts	восстановление БД из логического дампа

Таблица 3. Структура проекта.

2.3 Схема данных (Prisma)

Схема содержит модели Session, GpsPoint, Route, RouteCache, TrafficJob, AuditLog, ExportJob, BackupJob. Ключевые поля: `clientId` в Session (идемпотентность с `@@unique([deviceId, clientId])`), `deletedAt` (soft-delete), `payloadBytes` (контроль размера), `priority` и `scheduledFor` в TrafficJob для упорядоченной выборки Worker-ом.

prisma/schema.prisma — схема БД

```
model Session {
  id String @id @default(cuid())
  deviceId String
  clientId String? // UUID пакета для идемпотентности
  deviceName String?
  startTime DateTime
  endTime DateTime?
  pointCount Int @default(0)
  payloadBytes Int @default(0) // размер payload
  startLat Float?
  startLon Float?
  endLat Float?
  endLon Float?
  routeId String?
  planJson String? // JSON-поле плана маршрута
  deviationsJson String? // JSON отклонений
  deletedAt DateTime? // soft-delete timestamp (null = active)
  points GpsPoint[]
  route Route? @relation(fields: [routeId], references: [id])
  auditLogs AuditLog[] // записи аудита
  exportJobs ExportJob[] // задачи экспорта

  @@unique([deviceId, clientId]) // идемпотентность
  @@index([startTime(sort: Desc)]) // Список сессий по времени
  @@index([routeId]) // Поиск по избранному маршруту
  @@index([deletedAt]) // фильтр активных
}

model GpsPoint {
  id String @id @default(cuid())
  sessionId String
  lat Float
```

```
lon Float
speed Float?
altitude Float?
accuracy Float?
timestamp BigInt
bearing Float?
session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade)

@@index([sessionId]) // CRITICAL: WHERE sessionId = ?
@@index([sessionId, timestamp]) // ORDER BY внутри сессии
}

model Route {
  id String @id @default(cuid())
  name String
  description String?
  startLat Float
  startLon Float
  endLat Float
  endLon Float
  sessions Session[]
  routeCaches RouteCache[] // обратная связь для @relation в RouteCache
}

model RouteCache {
  id String @id @default(cuid())
  hash String @unique // hash(snap-to-grid start,end + tod_bucket)
  result String // JSON-результат маршрутизации
  todBucket Int // 0,3,6,9,12,15,18,21 (час)
  routeId String? // связь с избранным маршрутом
  expiresAt DateTime
  route Route? @relation(fields: [routeId], references: [id], onDelete: Cascade)

  @@index([todBucket, expiresAt]) // партиционированная инвалидация
  @@index([routeId]) // инвалидация по routeId
}

model TrafficJob {
  id String @id @default(cuid())
  sessionId String
  status String @default("pending") // pending|running|completed|failed
  attempts Int @default(0)
  priority Int @default(0) // выше = раньше
  scheduledFor DateTime @default(now()) // отложенный старт
  lockedBy String? // ID Worker-инстанса
  lockedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  error String? // Последняя ошибка при failed

  @@index([status, scheduledFor, priority]) // Очередь Worker-a
  @@index([sessionId])
}

// журнал аудита всех destructive-операций
// Связь с Session (onDelete: SetNull – audit сохраняется при удалении сессии)
model AuditLog {
  id String @id @default(cuid())
```

```

action String // session.delete|session.purge|session.export|route.delete|backup.*
targetId String // ID Session / Route / BackupJob
targetType String // Session | Route | BackupJob
actorType String // user | system | retention-cron | worker | backup-cron
actorId String? // userId / workerId / cron
metadata String? // JSON: детали (pointCount, format, reason, archivedUrl)
sessionId String? // FK на Session (nullable – аудит может относиться к Route/системе)
createdAt DateTime @default(now())
session Session? @relation(fields: [sessionId], references: [id], onDelete: SetNull)

@@index([action, createdAt]) // поиск по типу действия
@@index([targetId, targetType])
@@index([actorType, actorId])
@@index([sessionId]) // поиск аудита по сессии
}

// асинхронные задачи экспорта (для больших сессий)
model ExportJob {
id String @id @default(cuid())
sessionId String
format String // "gpx" | "kml" | "json"
status String @default("pending") // pending|running|completed|failed
fileUrl String? // временный URL (локальный / S3)
fileSize Int? // байты
expiresAt DateTime? // ссылка истекает через 24 ч
attempts Int @default(0)
lockedBy String? // ID Worker-инстанса
createdAt DateTime @default(now())
completedAt DateTime?
error String?
session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade)

@@index([status, createdAt])
@@index([sessionId])
}

// Модель для полного логического дампа БД
// Не переиспользуем ExportJob – он предназначен для одной сессии
model BackupJob {
id String @id @default(cuid())
type String // "full"
status String @default("pending") // pending|running|completed|failed|verified
fileUrl String? // путь к файлу дампа
fileSize Int? // байты
fileCount Int? // число экспортированных сессий
checksum String? // sha256 для верификации
attempts Int @default(0)
lockedBy String? // ID Worker-инстанса
createdAt DateTime @default(now())
completedAt DateTime?
verifiedAt DateTime? // время проверки целостности
error String?

@@index([status, createdAt])
@@index([type, createdAt])
}

```

Листинг 1. Схема БД.

2.4 Middleware

Единый middleware-слой (`src/middleware.ts`) обеспечивает сквозную обработку всех API-запросов. Шаг проверки размера payload (256 КБ для ingest) — запросы превышающие лимит отклоняются с HTTP 413 до попадания в route-handler, что защищает event loop от парсинга oversized JSON. Middleware выполняется до каждого route-handler и реализует: (а) **payload-size guard** — Content-Length проверка до чтения body; (б) авторизацию — проверка Bearer-токена или HttpOnly-куки; (в) rate limiting — sliding window через Redis (production) или in-memory (dev); (г) CORS; (д) security headers; (е) requestId; (ж) логирование method/path/duration/status.

`src/middleware.ts` — async function с импортами и try/catch

```
// src/middleware.ts — async function с импортами и try/catch
import { NextRequest, NextResponse } from "next/server";
import { checkRateLimit } from "@lib/rate-limit";
import { authenticate } from "@lib/auth";
import { corsResponse, setSecurityHeaders } from "@lib/http-utils";
import { json } from "@lib/http-utils";
import { logger } from "@lib/logger";

const MAX_PAYLOAD_BYTES = Number(process.env.MAX_PAYLOAD_BYTES ?? 262144);

export async function middleware(request: NextRequest): Promise {
  const requestId = crypto.randomUUID();
  const start = Date.now();

  try {
    // 0. payload-size guard (до чтения body)
    const cl = Number(request.headers.get("content-length") ?? 0);
    if (cl > MAX_PAYLOAD_BYTES) {
      return json({ error: "Payload too large", limit: MAX_PAYLOAD_BYTES }, 413);
    }

    // 1. CORS preflight
    if (request.method === "OPTIONS") return corsResponse(request);

    // 2. Rate limit check (Redis sliding window / in-memory LRU)
    const rateResult = await checkRateLimit(request);
    if (!rateResult.allowed) {
      return json(
        { error: "Rate limit exceeded", retryAfter: rateResult.retryAfter },
        429,
        { "X-RateLimit-Limit": String(rateResult.limit),
          "X-RateLimit-Remaining": String(rateResult.remaining),
          "X-RateLimit-Reset": String(rateResult.reset) }
      );
    }

    // 3. Authentication (Bearer token or HttpOnly cookie)
    const authResult = authenticate(request);
    if (!authResult.ok) {
      return json({ error: "Unauthorized" }, 401);
    }

    // 4. Security headers + requestId
    const response = NextResponse.next();
    setSecurityHeaders(response);
```

```
response.headers.set("X-Request-Id", requestId);
return response;
} catch (err) {
  logger.error("Middleware unexpected error", {
    requestId, method: request.method, path: request.nextUrl.pathname,
    error: err instanceof Error ? err.message : String(err),
    durationMs: Date.now() - start,
  });
  return json({ error: "Internal Server Error", requestId }, 500);
}
}
```

2.5 Контракты модулей

Каждый модуль бизнес-логики экспортирует формальный TypeScript-интерфейс, что обеспечивает контракт между модулями и возможность мокирования в тестах. DI-инъекция через фабричные функции. Контракт `IIdeмпotencyService` обеспечивает проверку (`deviceId`, `clientId`), `IRateLimiter` — backend-agnostic rate limiting (Redis/in-memory).

src/lib/contracts.ts

```
interface IRoutingService {
  route(start: Coord, end: Coord, todBucket?: number): Promise;
  getProviders(): string[]; // ["2gis", "osrm", "haversine"]
}

interface ITrafficService {
  fetchForSegments(segments: Segment[], utc: number): Promise;
  getConcurrency(): number; // 3-5 параллельных
}

interface IPlanService {
  build(sessionId: string, target: PlanTarget): Promise;
  computeDeviations(plan: Plan, points: GpsPoint[]): Deviation[];
}

interface ICacheService {
  get(key: string): Promise;
  set(key: string, value: unknown, ttlMs: number): Promise;
  invalidate(routeId: string): Promise;
}

// Контракт идемпотентности
interface IIdeмпotencyService {
  check(deviceId: string, clientId: string): Promise<{ sessionId: string; duplicate: boolean }>;
}

// Контракт идемпотентности
interface IRateLimiter {
  check(key: string, limit: number, windowSec: number): Promise<{ allowed: boolean; remaining: number; retryAfter: number }>;
}

// Фабрики с DI
function createRoutingService(cache?: ICacheService): IRoutingService;
function createTrafficService(logger?: ILogger): ITrafficService;
```

```
function createIdempotencyService(db: PrismaClient): IIdempotencyService;
function createRateLimiter(backend: "redis" | "memory"): IRateLimiter;
```

2.6 Изоляция Worker-процесса

Изоляция Worker-процесса от API event loop

TrafficJob обрабатывается в отдельном Worker-процессе. Если бы обработка шла in-process (в том же Node.js-процессе, что и API), при 100 сессиях/мин ($100 \times \sim 50$ сегментов = 5000 fetch) event loop API-сервера забивался бы сетевыми I/O, и ingest переставал отвечать. Выделение Worker-процесса решает эту проблему.

API-сервер и Worker-процесс — два независимых процесса. API-хендлер `POST /api/ingest` выполняет только синхронные быстрые операции: Zod-валидация, проверка идемпотентности, `INSERT GPS-точек` (одна транзакция), `INSERT строки в TrafficJob` со статусом `pending`. Ответ клиенту возвращается немедленно (HTTP 201 за ≤ 50 мс типично), после чего event loop свободен для следующего ingest-запроса.

Worker-процесс (`worker/index.ts`) — отдельный Node.js-процесс, запускаемый параллельно с API-сервером (Docker compose: два сервиса; Vercel: serverless-функция по Cron). Worker опрашивает таблицу TrafficJob с интервалом `WORKER_POLL_INTERVAL_MS` (по умолчанию 5 с), выбирает пачку pending-задач (`LIMIT 10`), атомарно захватывает их через `UPDATE ... SET status='running', lockedBy=$workerId, lockedAt=now() WHERE id IN (...) AND status='pending' RETURNING id` и обрабатывает только те id, что вернулись в `RETURNING` (см. ниже про верификацию). Controlled concurrency внутри Worker: `p-limit(5)` — не более 5 параллельных traffic-fetch.

Шаг	API-сервер	Worker-процесс
1	Принимает ingest, Zod-валидация	—
2	Проверка идемпотентности (deviceId, clientId)	—
3	INSERT GpsPoint[] (транзакция)	—
4	INSERT TrafficJob(status=pending)	—
5	HTTP 201 → клиенту (≤ 50 мс)	—
6	—	Опрос: SELECT pending LIMIT 10 (каждые 5 с)
7	—	UPDATE ... SET running, lockedBy (атомарный захват)
8	—	fetch traffic (2ГИС), p-limit(5), retry×3, backoff
9	—	UPDATE TrafficJob SET status=completed, result=...
10	—	Если failed: attempts++, backoff, повтор при attempts < 3

Таблица 4. Разделение ответственности API-сервер ↔ Worker-процесс.

В Docker-окружении API и Worker запускаются как два сервиса в одном `docker-compose.yml` с общей переменной `DATABASE_URL`. В Vercel (serverless) Worker реализуется как отдельная serverless-функция `/api/cron/worker`, вызываемая Vercel Cron каждые 5 минут (минимальный

интервал Vercel Cron на free-плане). На Pro-плане минимальный интервал — 1 минута; это рекомендуемая конфигурация для целевой нагрузки 100 сессий/мин (см. §9.6 — free-план классифицирован как BLOCKING). Для self-hosted Docker рекомендуется постоянный Worker-процесс с `restart: always`.

Горизонтальное масштабирование Worker

При росте очереди Worker-процессы можно масштабировать горизонтально: запустить N инстансов Worker с разными `WORKER_ID`. Атомарный захват (`UPDATE ... WHERE status='pending' RETURNING id`) гарантирует, что одна задача не будет обработана двумя Worker-ами одновременно. Рекомендация: 1 Worker на каждые 50 сессий/мин (эмпирически выведено из k6-тестов, см. §8.4).

Верификация атомарного захвата через RETURNING

При репликации Turso (read-replica) возможна ситуация: Worker A читает pending-задачу из реплики, Worker B из другой реплики видит ту же задачу как pending (задержка репликации ~50-500 мс). Оба вызывают `UPDATE`, но только первый успешно меняет status. Верификация через `RETURNING id`: Worker обрабатывает только те id, которые фактически вернул `UPDATE` (то есть изменённые этим Worker-ом). Если `RETURNING` пустой — задачу уже захватил другой Worker, пропуск. Для LibSQL/Turso также рекомендуется `BEGIN IMMEDIATE` для write-транзакций захвата, что берёт write-lock на primary до любых `SELECT/UPDATE`. См. §3.9 — ограничения масштабирования.

worker/processor.ts — RETURNING + Promise.all + импорты

```
// worker/processor.ts – верифицируемый атомарный захват через RETURNING
import { db } from "../src/lib/db";
import { randomUUID } from "node:crypto";
import pLimit from "p-limit"; // импорт для concurrency control

const WORKER_ID = process.env.WORKER_ID ?? randomUUID();
const BATCH_SIZE = Number(process.env.WORKER_BATCH_SIZE ?? 10);

async function pollAndClaim(): Promise {
  // SELECT кандидатов (можно из реплики)
  const candidates = await db.$queryRaw<{ id: string }[]>`
  SELECT id FROM TrafficJob
  WHERE status = 'pending' AND scheduledFor <= datetime('now')
  ORDER BY priority DESC, scheduledFor ASC
  LIMIT ${BATCH_SIZE}
  `;
  if (candidates.length === 0) return [];
  const ids = candidates.map((c) => c.id);

  // АТОМАРНЫЙ ЗАХВАТ с верификацией через RETURNING
  // Применяется к primary (write-traffic идёт на primary, не на реплику)
  const claimed = await db.$queryRaw<{ id: string }[]>`
  UPDATE TrafficJob
  SET status = 'running', lockedBy = ${WORKER_ID}, lockedAt = datetime('now')
  WHERE id IN (${ids.join(",")})
  AND status = 'pending'
  AND lockedBy IS NULL
  RETURNING id
  `;
  // Только реально захваченные этим Worker-ом id
  return claimed.map((c) => c.id);
}
```

```
}

async function main() {
while (true) {
try {
const claimedIds = await pollAndClaim();
if (claimedIds.length > 0) {
// Обрабатываем только захваченные – p-limit(5) для concurrency
// p-limit не имеет метода .map – используем Promise.all + limit()
const limit = pLimit(5);
await Promise.all(claimedIds.map((id) => limit(() => processJob(id))));
} else {
await sleep(Number(process.env.WORKER_POLL_INTERVAL_MS ?? 5000));
}
} catch (err) {
logger.error("Worker poll failed", { workerId: WORKER_ID, error: String(err) });
await sleep(5000);
}
}
}
```


3. Принципы работы

3.1 Сбор GPS-данных (правки #5, #6)

Данные поступают через `POST /api/ingest`. Поток обработки усилен двумя механизмами: **идемпотентностью** (`clientId`) и **payload-лимитом** (`MAX_PAYLOAD_BYTES` + `p-limit`). Полный конвейер:

- Проверка `Content-Length` ≤ 256 КБ в middleware $\rightarrow$ HTTP 413 при превышении (до чтения body).
- Zod-валидация пакета: `lat` $\in [-90, 90]$, `lon` $\in [-180, 180]$, `speed` $\in [0, 83.33]$ м/с, ≤ 1000 точек, **`clientId` — обязательный UUID/cuid**.
- Нормализация таймстемпов: `nanosec` $\rightarrow$ `msec` (деление на 1 000 000), фильтрация точек с `gap` > 30 с.
- **Идемпотентность**: `SELECT Session WHERE deviceId=? AND clientId=?` — при существовании возвращается HTTP 200 + старый `sessionId` без повторной записи.
- Сериализация DB-write через `p-limit(1)` — protects SQLite write lock; параллельные ingest-запросы выстраиваются в очередь вместо конкуренции за lock.
- `INSERT GpsPoint[]` + `INSERT TrafficJob(pending)` в одной транзакции.
- Ответ: HTTP 201 { `sessionId`, `pointsAccepted`, `trafficJobId`, `duplicate: false` } для нового пакета, или HTTP 200 { `sessionId`, `duplicate: true` } для повторной отправки.

Валидация гарантирует: не более 1000 точек в пакете, `deviceId` длиной 1-64 символа, `clientId` — валидный UUID v4 или cuid. При невалидном запросе возвращается HTTP 400 с JSON: { `error: "Validation failed", field: "lat", message: "Must be between -90 and 90"` }. Размер body проверяется дважды: сначала по заголовку `Content-Length` в middleware (быстрый путь), затем по фактическому размеру после парсинга (защита от подделанного заголовка).

3.2 Цепочка маршрутизации

Трёхуровневая цепочка с автоматическим fallback. Первичный провайдер — 2ГИС (carrouting 6.0.0), возвращает детализированный маршрут с полилинией, сегментами и манёврами. При недоступности — OSRM через demo-сервер. Третий уровень — гаверсинус (40 км/ч). Перед вызовом API проверяется кэш (LRU + SQLite): ключ — `hash(startLat, startLon, endLat, endLon snap-to-grid ~55 м + todBucket)`, при hit возвращается закешированный результат без вызова API. При miss — вызов API, результат сохраняется в кэш (TTL: 5 мин для LRU, 24 ч для persistent). Fallback-события логируются и инкрементируют метрику `routing_fallback_total`.

3.3 Исторические пробки (Traffic) — обработка в Worker

После ingest API создаёт `TrafficJob` (status: pending) и сразу возвращает ответ клиенту. Дальнейшая обработка полностью происходит в **Worker-процессе** (см. §2.6): Worker опрашивает очередь, захватывает pending-задачи и выполняет traffic-fetch с controlled concurrency: `p-limit(5)` параллельных запросов к 2ГИС traffic-endpoint, с задержкой 200-500 мс между батчами. `Promise.allSettled` используется вместо `Promise.all` — при ошибке одного сегмента остальные продолжают. Retry-логика: 3 попытки с exponential backoff (1 с, 2 с, 4 с). Статус обновляется: pending $\rightarrow$ running $\rightarrow$ completed/failed. При failed — ошибка сохраняется в `TrafficJob.error`, attempts инкрементируется; при attempts ≥ 3 задача переходит в статус **dead** и требует ручного вмешательства.

Результат обогащает сегменты плана полями: `trafficFetched`, `trafficUtc`, `trafficSpeedKmh`, `trafficDurationSec`, `trafficSource`.

Почему Worker решает проблему event loop

Worker-процесс изолирован от API-сервера: даже если его event loop полностью загружен fetch-ами, API-сервер продолжает принимать ingest с $p95 < 50$ мс. Разделение процессов = разделение event loop-ов.

3.4 План-фактный анализ

Центральная функция сервиса. План строится при создании сессии или по запросу. Система определяет стартовую точку (первая GPS-точка) и конечную (последняя или координаты из Route). Маршрут сегментируется по манёврам. Для каждого сегмента вычисляются плановые метрики: дистанция, ожидаемая длительность, ожидаемая скорость. Затем GPS-данные сопоставляются с сегментами (по географической близости к полилинии), и вычисляются фактические скорость и длительность. Результат — таблица отклонений (`segmentDeviations`) с полями: `planSpeedKmh`, `actualSpeedKmh`, `speedDeviation`, `planDurationSec`, `actualDurationSec`, `durationDeviation`, а также `traffic`-поля.

3.5 Кэширование маршрутов

Архитектура кэш-ключа маршрутизации

Кэш-ключ строится на основе snap-to-grid координат и time-of-day bucket.

Обоснование snap-to-grid

Snap-to-grid ($\text{Math.round}(v / \text{GRID_SIZE}) * \text{GRID_SIZE}$) с $\text{GRID_SIZE}=0.0005^\circ$ даёт ячейку ~ 55 м (lat) $\times$ ~ 35 м (lon). Это превышает точность GPS-приёмника iPhone (5-10 м), делая ключ устойчивым к естественному GPS-шуму: две точки в пределах одной ячейки дают один ключ, две точки в разных ячейках (расстояние > 35 м) — разные ключи. Ожидаемый hit ratio — 70-85% (требует подтверждения в k6-тестах, см. §8.4).

Кэш-ключ состоит из двух компонентов. **Первый** — **snap-to-grid** координат до сетки с шагом $\text{GRID_SIZE} = 0.0005^\circ$. На широте Саратова ($\sim 51^\circ$): 1° широты $\approx 111\,200$ м (почти постоянно), 1° долготы $\approx 111\,320 \cdot \cos(51^\circ) \approx 70\,000$ м. Следовательно $\text{GRID_SIZE} = 0.0005^\circ$ даёт ячейку ~ 55 м (по широте) $\times$ ~ 35 м (по долготе). **Второй** компонент — **time-of-day bucket**: час дня округляется вниз до кратного 3 (8 бакетов: 0, 3, 6, 9, 12, 15, 18, 21 UTC). Маршрут «дом → работа» в 08:00 и в 20:00 получает разные кэш-записи с разными traffic-данными.

Двухуровневый кэш реализован в `src/lib/cache.ts`. Уровень 1: in-memory LRU (Map + doubly-linked list, макс. 100 записей, TTL 5 мин). Уровень 2: persistent в таблице RouteCache (hash, result, todBucket, routeId, expiresAt, TTL 24 ч). Паттерн cache-aside: сначала LRU, при miss — RouteCache, при miss — вызов API, результат пишется в оба уровня.

`src/lib/cache.ts` — snap-to-grid, корректная геометрия lat/lon

```
// src/lib/cache.ts – snap-to-grid вместо toFixed
import { createHash } from "node:crypto";
import type { Coord } from "../types";
import { db } from "../db";
```

```
// snap-to-grid вместо toFixed(precision).
// Геометрия (на широте Саратова ~51°):
// Для широты: 1° ≈ 111 200 м (почти постоянно на всех широтах).
// GRID_SIZE=0.0005 → 111 200 * 0.0005 ≈ 55.6 м.
// Для долготы: 1° ≈ 111 320 * cos(lat) м.
// На широте 51°: 111 320 * cos(51°) ≈ 111 320 * 0.6293 ≈ 70 046 м.
// GRID_SIZE=0.0005 → 70 046 * 0.0005 ≈ 35.0 м.
// Итог: ячейка сетки ~55 м (lat) × ~35 м (lon).
// Устойчивость к GPS-шуму (5-10 м): 2 точки в одной ячейке → один ключ.
// Различение маршрутов: точки в разных ячейках (расстояние > 35 м) → разные ключи.
const GRID_SIZE = 0.0005;
// 8 трёхчасовых бакетов: пробки в час пик ≠ ночные
const TOD_BUCKETS = [0, 3, 6, 9, 12, 15, 18, 21] as const;

// Округление до центра ячейки сетки (snap-to-grid)
function snap(v: number): number {
  return Math.round(v / GRID_SIZE) * GRID_SIZE;
}

export function buildCacheKey(start: Coord, end: Coord, ts: Date): string {
  const s = `${snap(start.lat)},${snap(start.lon)}`;
  const e = `${snap(end.lat)},${snap(end.lon)}`;
  // bucket = максимальный бакет, не превышающий текущий час
  const bucket = TOD_BUCKETS.filter((b) => b <= ts.getUTCHours()).pop() ?? 0;
  const raw = `${s}|${e}|${bucket}`;
  return createHash("sha256").update(raw).digest("hex");
}

// Инвалидация: по routeId (изменение избранного маршрута)
export async function invalidateRoute(routeId: string): Promise {
  await db.routeCache.deleteMany({
    where: { routeId }, // поле routeId в RouteCache (см. §2.3)
  });
}

// TTL: LRU 5 мин, persistent 24 ч. При todBucket-переходе автоматически
// становится miss (другой ключ) — ручная инвалидация по времени не нужна.
```

Параметр	Значение	Обоснование
Метод округления	snap-to-grid (Math.round(v / GRID_SIZE) * GRID_SIZE)	Устойчив к GPS-шуму 5-10 м
GRID_SIZE	0.0005°	~55 м (lat) × ~35 м (lon) на широте Саратова
Разрешение ключа	~55 м × ~35 м	Ячейка > GPS-шум, различает маршруты между кварталами
Hit ratio (ожидаемый)	70–85% (требует подтверждения в CI)	Эмпирическая оценка, не подтверждена к6-тестами
Time-of-day в ключе	8 бакетов по 3 ч (0,3,6,9,12,15,18,21 UTC)	Пробки в час пик ≠ ночные
TTL LRU	5 мин	Быстрый кэш для горячих маршрутов
TTL persistent	24 ч (партиционировано по todBucket)	Нет устаревших пробок при смене времени суток

Параметр	Значение	Обоснование
routeId в RouteCache	да (поле + @relation)	Инвалидация по избранному маршруту
Инвалидация по routeId	да (deleteMany where routeId)	При изменении/удалении избранного маршрута
Инвалидация по времени	авто (смена todBucket)	Новый бакет = новый ключ = miss

Таблица 5. Параметры кэширования маршрутов.

3.6 Идемпотентность ingest

Идемпотентность через clientId

Sensor Logger при сетевом сбое ретранслирует пакет. Без идемпотентности повторная отправка создавала бы новую сессию с дублированными GPS-точками — данные портились, segmentDeviations считались по «фантомной» сессии.

В IngestSchema определено поле `clientId` — UUID v4, генерируемый клиентом Sensor Logger на стороне устройства перед каждой отправкой. При повторной передаче (из-за сетевого таймаута, 5xx ответа, потери пакета) **тот же** `clientId` отправляется повторно. Сервер проверяет пару (`deviceId`, `clientId`) по UNIQUE-индексу:

- **Новый пакет:** INSERT успешен → HTTP 201, `pointsAccepted`, `trafficJobId`, `duplicate=false`.
- **Повтор пакета:** UNIQUE constraint violation → catch → SELECT существующего Session → HTTP 200, `duplicate=true`, тот же `sessionId`. Не создаётся ни новой сессии, ни дублирующих точек, ни нового TrafficJob.
- **Конкуренция:** если два одинаковых запроса приходят одновременно (split-brain сети), UNIQUE-индекс БД гарантированно пропускает только первый; второй получает constraint violation и fallback на SELECT.

Для clients, не способных генерировать `clientId` (legacy Sensor Logger), поле объявлено опциональным (`clientId?: string`); в этом случае идемпотентность не гарантируется, и в лог пишется warn "ingest without clientId – idempotency disabled". После 100% обновления клиентов поле станет обязательным.

3.7 Политика хранения данных — 10 лет

Политика хранения 10 лет с ежедневным cron-очистением

Без retention policy БД растёт без ограничения: через 3-5 лет активного использования — сотни МБ, деградация индексов, медленная пагинация в GET /api/sessions. Формальная политика хранения 10 лет с ежедневным cron-очистением решает эту проблему.

Введена **политика хранения 10 лет**. Значение по умолчанию `RETENTION_DAYS = 3650` (10×365), конфигурируется через env. Обоснование выбора 10 лет: (а) соответствует максимальному сроку хранения персональных данных по законодательству РФ для личной аналитики; (б) GDPR right-to-be-forgotten закрывается через DELETE /api/sessions/[id] (см. §4.10) — пользователь может удалить данные раньше; (в) $10 \text{ лет} \times 365 \text{ дней} \times 1 \text{ сессия/день} \times \sim 100 \text{ КБ} = \sim 365 \text{ МБ}$ — укладывается в Turso free tier (9 ГБ) с запасом $\times 25$; (г) достаточно для долгосрочной аналитики «как изменился маршрут дом-работа за десятилетие».

Исполнение политики — ежедневная cron-задача в Worker-процессе. Worker запускает retention-процедуру раз в сутки в фиксированное время (по умолчанию 00:30 UTC, конфигурируется через `RETENTION_CRON_UTC`). Алгоритм:

Шаг	Действие	SQL / код
1	SELECT кандидатов на удаление	SELECT id, endTime, pointCount FROM Session WHERE endTime < datetime('now', '-' \$RETENTION_DAYS ' days') AND deletedAt IS NULL
2	Для каждой: авто-архивация (если включено)	Создать ExportJob(format=json), дождаться completed, получить archivedUrl
3	Транзакция: hard delete Session	DELETE FROM Session WHERE id = ? (cascade: GpsPoint, TrafficJob, ExportJob через onDelete: Cascade)
4	Запись в AuditLog	INSERT INTO AuditLog(action="session.purge", targetId=?, actorType="retention-cron", metadata={archivedUrl, pointCount, ageDays})
5	Инкремент метрик	retention_purge_total{archived=true false}++, retention_purge_age_seconds.observe(ageDays)

Таблица 5.1. Алгоритм retention-cron (ежедневно, 00:30 UTC). SQL совместим с SQLite/Turso.

SQLite-совместимый синтаксис арифметики дат

Используется нативный SQLite-синтаксис `datetime('now', '-' || $RETENTION_DAYS || ' days')` для арифметики дат. При будущей миграции на PostgreSQL (см. §3.9) запросы выносятся в адаптер `src/lib/db-adapter.ts` или используются Prisma-методы вместо сырого SQL.

Архивация (`RETENTION_ARCHIVE_ENABLED = true` по умолчанию) перед удалением: создаётся ExportJob в формате JSON (полный дамп Session + GpsPoint[] + plan + deviations), файл сохраняется в `EXPORT_STORAGE_DIR` (локальная директория или S3-compatible bucket), URL записывается в AuditLog.metadata.archivedUrl. При `RETENTION_ARCHIVE_ENABLED = false` — удаление без архивации (для GDPR-режима «forget me полностью»). Срок хранения архивов — `ARCHIVE_RETENTION_DAYS = 3650` (те же 10 лет), после чего файлы удаляются отдельной cron-задачей.

Индивидуальные override-ы retention

Поддерживается per-session override через поле `retentionOverrideDays` (Int?, nullable). Если установлено — сессия удаляется по этому сроку, а не по глобальному. Используется для: сессий, помеченных пользователем как «важные» (keep forever = NULL или 36500), или «неважные» (ускоренное удаление = 30 дней). Поле добавляется миграцией expand-contract, по умолчанию NULL (использовать глобальный RETENTION_DAYS).

Soft-delete (поле `deletedAt` в Session) используется для «корзины»: при DELETE /api/sessions/[id] (см. §4.10) устанавливается `deletedAt = now()`, сессия скрывается из списков по умолчанию. Через 30 дней grace period другая cron-задача (GRACE_PERIOD_DAYS = 30) выполняет hard delete с cascade. В течение grace period пользователь может восстановить сессию через POST /api/sessions/[id]/restore (убирает deletedAt). Если `deletedAt` уже установлено при срабатывании retention-cron — сессия удаляется сразу, без повторной архивации.

3.7.1 Влияние на производительность

Retention-cron обрабатывает не более 100 сессий за один запуск (LIMIT 100), между итерациями пауза 2 секунды для снижения нагрузки на БД. При большом объёме помеченных к удалению (например, первый запуск после 11 лет эксплуатации) — обработка растягивается на несколько дней, что допустимо (нет SLA на удаление). Worker помечает обработанные сессии через `deletedAt = now()` перед hard delete, что предотвращает их повторный SELECT в следующей итерации. Индекс @@index([deletedAt]) обеспечивает быстрый фильтр активных.

3.8 Политика деградации

Трёхуровневая цепочка с уведомлением пользователя

При длительном отказе 2ГИС сервис переключается на OSRM demo (без SLA) и гаверсинус. Пользователь уведомляется о снижении качества через badge и поле degraded в API-ответе.

Трёхуровневая цепочка маршрутизации (2ГИС → OSRM → гаверсинус) описана в §3.2. Каждый fallback сопровождается: (а) **метрикой** routing_fallback_total с label provider; (б) **логированием warn** с requestId/sessionId; (в) **уведомлением пользователя** через поле в ответе API и badge в веб-интерфейсе. Это закрывает проблему «тихого падения качества».

Уровень	Провайдер	SLA	Уведомление пользователя	Метрика
1 (primary)	2ГИС carrouting 6.0.0	99.5% uptime, p95 < 2 с	Нет (нормальный режим)	routing_provider_total{provider="2gis"}
2 (fallback)	OSRM demo server	Без SLA, best-effort	Жёлтый badge «Маршрут: OSRM (2ГИС недоступен)»	routing_fallback_total{provider="osrm"}
3 (last resort)	Гаверсинус (40 км/ч)	Всегда доступен (локальный расчёт)	Красный badge «Маршрут: приближённый расчёт (нет данных о пробках)»	routing_fallback_total{provider="haversine"}

Таблица 5.2. Политика деградации цепочки маршрутизации.

В API-ответе (GET /api/plan/[sessionId], POST /api/plan) добавляется поле `routingProvider` и `degraded: boolean`. При `degraded=true` фронтенд показывает badge в PlanDialog и над картой. Дополнительно: если 2ГИС недоступен более 5 минут (метрика `routing_fallback_total > 50%` за 5 мин) — алерт в лог (error) и уведомление оператору через /health endpoint (`routing2gis: false, status: "degraded"`). Пользователь видит banner «Сервис 2ГИС временно недоступен, маршруты строятся по резервной цепочке» до восстановления.

Circuit Breaker для 2ГИС

При 5 последовательных отказах 2ГИС (timeout / 5xx) в течение 1 минуты — circuit breaker размыкается на 30 секунд: все запросы сразу идут на OSRM без попытки 2ГИС. Это предотвращает «лавину» timeout-ов при длительном отказе. После 30 с — half-open: один пробный запрос к 2ГИС, при успехе — замыкание (возврат к primary). Реализация: `src/lib/circuit-breaker.ts`, состояние in-memory (single-instance) или Redis (multi-instance).

3.9 Ограничения масштабирования

Single-writer архитектура SQLite и порог миграции

Система заявляет горизонтальное масштабирование (несколько Worker, multi-instance rate-limit через Redis), но БД — SQLite/Turso с единым писателем и `p-limit(1)` на `DB-write`. Это архитектурный потолок: масштабирование упирается в одну точку записи. Формализованы потолки и порог миграции на PostgreSQL.

SQLite (и его managed-версия Turso/LibSQL) — single-writer архитектура. Все write-операции сериализуются через write-lock на уровне файла/БД. Это явно учтено: ingest DB-write сериализован через `p-limit(1)`, что превращает конкуренцию за lock в упорядоченную очередь. Это даёт предсказуемую латентность, но ставит **жёсткий потолок пропускной способности записи**.

Ресурс	Потолок (single-instance)	Причина	Порог миграции
Write throughput (ingest)	~200 сессий/мин	p-limit(1) + SQLite write-lock, эмпирически из k6	> 150 сессий/мин sustained
Read throughput (sessions list)	~2000 req/мин	Read-реплики Turso масштабируют	> 5000 req/мин
Worker-инстансов	безл. (горизонтальное)	Каждый со своим WORKER_ID, RETURNING-захват	—
API-инстансов	безл. (stateless)	Stateless, rate-limit через Redis	—
Одновременных пользователей	~500 активных	Session списки, Cursor-based пагинация	> 1000 MAU

Таблица 5.3. Потолки масштабирования и пороги миграции.

Порог миграции на PostgreSQL: при достижении любого из: (а) sustained ingest > 150 сессий/мин в течение 1 часа; (б) p99 ingest latency > 500 мс; (в) > 1000 MAU. Миграция предполагает: смену DATABASE_URL на PostgreSQL, обновление Prisma schema (минимальные изменения — Prisma абстрагирует диалект), удаление p-limit(1) на ingest (PostgreSQL поддерживает MVCC concurrent writes),

возможно partitioning таблицы GpsPoint по sessionId для оптимизации больших сессий. Оценка трудозатрат: 3-5 дней + expand-contract миграция данных. См. §9.9 — план миграции.

Текущая архитектура — оптимизирована для personal-scale

Система спроектирована для 1 пользователя (владелец + семья). 100 сессий/мин — это пиковая нагрузка (например, массовая синхронизация Sensor Logger после долгого offline), не sustained. Для этого сценария SQLite/Turso + p-limit(1) — оптимально: простота развёртывания, нулевые операционные затраты, backup через Turso snapshots. Миграция на PostgreSQL — premature optimization для текущей аудитории, но порог и план зафиксированы.

4. API-эндпоинты

4.1 POST /api/ingest (правки #5, #6)

Приём GPS-данных от мобильного клиента. Авторизация: Bearer INGEST_TOKEN. Zod-валидация пакета (см. §4.7). При успехе: нормализация, фильтрация по gap, проверка идемпотентности, запись в БД (p-limit(1) serialization), создание TrafficJob. Ответ: HTTP 201 { sessionId, pointsAccepted, trafficJobId, duplicate: false } для нового пакета или HTTP 200 { sessionId, duplicate: true } для повторной отправки. При невалидных данных: HTTP 400 { error, field, message }. При превышении payload: HTTP 413. При превышении rate limit: HTTP 429.

POST /api/ingest — запрос и ответы

```
// Пример запроса – clientId для идемпотентности
{
  "deviceId": "iphone-se-xxx",
  "clientId": "550e8400-e29b-41d4-a716-446655440000", // UUID v4
  "points": [{
    "lat": 51.53, "lon": 46.0, "speed": 12.5,
    "altitude": 160, "accuracy": 5.0,
    "timestamp": 1723680000000000000,
    "bearing": 45
  }]
}

// Пример ответа (новый пакет)
201 Created
{
  "sessionId": "clxyz...",
  "pointsAccepted": 47,
  "trafficJobId": "clabc...",
  "duplicate": false
}

// Пример ответа (повтор пакета – идемпотентность)
200 OK
{
  "sessionId": "clxyz...", // тот же, что и в первом ответе
  "duplicate": true
}
```

4.2 GET /api/sessions — фильтры

Список сессий в обратном хронологическом порядке. Cursor-based пагинация: ?cursor=xxx&limit;=20. Ответ: { sessions, nextCursor }. Авторизация: HttpOnly-кука или Bearer API_KEY (server-side). Поддерживаются фильтры по дате и маршруту для управления большими объёмами данных (10К+ сессий за 10 лет):

Параметр	Тип	Назначение	Пример
cursor	string (cuid)	Курсор последней сессии предыдущей страницы	?cursor=clxyz123
limit	int (1-100)	Размер страницы, по умолчанию 20	?limit=50

Параметр	Тип	Назначение	Пример
olderThan	ISO 8601 datetime	Только сессии с endTime < olderThan (исключая)	?olderThan=2025-01-01T00:00:00Z
before	ISO 8601 datetime	Только сессии с endTime > before (новее)	?before=2025-12-31T23:59:59Z
routeId	string (cuid)	Только сессии по конкретному избранному маршруту	?routeId=clabc456
includeDeleted	boolean	Включать soft-deleted (deletedAt != null). По умолчанию false	?includeDeleted=true
minPoints	int	Только сессии с pointCount >= minPoints (отсев фантомов)	?minPoints=10

Таблица 6.1. Query-параметры GET /api/sessions.

Все параметры опциональны и комбинируются. Backward-compatible: вызовы без новых параметров работают без новых параметров. Пример комбинированного запроса: GET /api/sessions?olderThan=2025-01-01T00:00:00Z&routeId=clabc456&limit=50 — вернёт до 50 сессий по маршруту clabc456, завершённых до 1 января 2025 года. Фильтр по deletedAt IS NULL применяется всегда, если includeDeleted не true. Для пагинации в сочетании с фильтрами: cursor кодирует (endTime, id), при изменении фильтров курсор сбрасывается.

4.3 GET /api/sessions/[id]

Полная информация о сессии. Cursor-based пагинация GPS-точек: ?pointsCursor=xxx&pointsLimit=500. Ответ: { session, points, nextPointsCursor, segmentDeviations }. Пагинация предотвращает загрузку 10K+ точек за один запрос.

4.4 POST /api/plan

Создание плана маршрута. Параметры: routeId (избранный маршрут) или endLat+endLon. Zod-валидация: routeId — string cuid, endLat/endLon — number в диапазонах. Запускает цепочку маршрутизации (2ГИС → OSRM → гаверсинус), сегментирует, сохраняет план, создаёт TrafficJob (pending). Ответ: HTTP 201 { plan, trafficJobId }. План и TrafficJob создаются в одной транзакции.

4.5 GET /api/plan/[sessionId]

План маршрута для сессии. Включает сегменты с traffic-полями. Если TrafficJob ещё pending/running — в ответе поле trafficStatus: "pending" | "running", фронтенд показывает skeleton и опрашивает с интервалом 3 с.

4.6 CRUD /api/routes

Управление избранными маршрутами: GET (список), POST (создание/upsert), DELETE (по ID). Zod-валидация координат. При DELETE инвалидируются все записи RouteCache, связанные с этим маршрутом (по routeId, см. §3.5).

4.7 Валидация входных данных

Все API endpoints защищены Zod-схемами, определёнными в `src/lib/validation.ts`. Валидация выполняется до любого взаимодействия с БД. `IngestSchema` расширена обязательным полем `clientId` (с fallback-режимом для legacy-клиентов).

`src/lib/validation.ts` — Zod-схемы

```
import { z } from "zod";

export const PointSchema = z.object({
  lat: z.number().min(-90).max(90),
  lon: z.number().min(-180).max(180),
  speed: z.number().min(0).max(83.33).optional(), // 300 км/ч
  altitude: z.number().optional(),
  accuracy: z.number().positive().optional(),
  timestamp: z.bigint().positive(),
  bearing: z.number().min(0).max(360).optional(),
});

// clientId (optional для legacy, будет required после миграции клиентов)
export const IngestSchema = z.object({
  deviceId: z.string().min(1).max(64),
  clientId: z.string().uuid().optional(), // идемпотентность
  points: z.array(PointSchema).min(1).max(1000),
}).refine(d => d.points.length <= 1000, {
  message: "Max 1000 points per ingest",
});

export const PlanSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("route"), routeId: z.string().cuid() }),
  z.object({ type: z.literal("coords"),
    endLat: z.number().min(-90).max(90),
    endLon: z.number().min(-180).max(180) }),
]);

export const RouteSchema = z.object({
  name: z.string().min(1).max(128),
  description: z.string().max(512).optional(),
  startLat: z.number().min(-90).max(90),
  startLon: z.number().min(-180).max(180),
  endLat: z.number().min(-90).max(90),
  endLon: z.number().min(-180).max(180),
});
```

4.8 GET /health

Health-check endpoint для оркестрации и мониторинга. Проверяет: (а) доступность БД (SELECT 1 через Prisma), (б) доступность 2ГИС API (HEAD-запрос, таймаут 5 с), (в) **глубину очереди TrafficJob** (COUNT WHERE status=pending, warn если > 100), (г) использование памяти (process.memoryUsage()).
Ответ: HTTP 200 { status: "ok", db: true, routing2gis: true, trafficQueueDepth: 3, memoryMb: 128 } или HTTP 503 { status: "degraded", db: true, routing2gis: false, trafficQueueDepth: 542, memoryMb: 512 }. Docker HEALTHCHECK и Kubernetes livenessProbe настраиваются на этот endpoint.

4.9 Worker API — внутренние endpoints

Worker-процесс взаимодействует с API-сервером (или напрямую с БД) через выделенные внутренние endpoints. Все они защищены `CRON_SECRET` (отдельный токен, не `INGEST_TOKEN`). При Docker-деплое Worker обращается к БД напрямую (общий `DATABASE_URL`); при Vercel — через эти endpoints.

Endpoint	Метод	Назначение	Авторизация
/api/worker/poll	POST	Выбрать pending-задачи (LIMIT 10), атомарно захватить	CRON_SECRET
/api/worker/complete	POST	Отметить задачу completed, записать result JSON	CRON_SECRET
/api/worker/fail	POST	Отметить failed, инкремент attempts, сохранить error	CRON_SECRET
/api/worker/stats	GET	Метрики очереди: pending/running/failed/dead счётчики	CRON_SECRET
/api/worker/requeue	POST	Ручной requeue dead-задач (админка)	ADMIN_TOKEN

Таблица 6. Внутренние Worker-API endpoints.

4.10 DELETE /api/sessions/[id]

DELETE с soft-delete, grace period и audit log

DELETE endpoint для Session: soft-delete с grace period 30 дней, audit log всех операций. Пользователь может удалить поездку (приватность, фантомные записи, GDPR right-to-be-forgotten).

Удаление сессии по ID. Авторизация: HttpOnly cookie (веб-клиент) или Bearer API_KEY. Реализован двухфазный паттерн: **soft-delete** (установка deletedAt) + **hard-delete** через grace period (30 дней) отдельной cron-задачей. Это даёт пользователю возможность «отменить» удаление в течение месяца — типичная UX-практика «корзины».

DELETE /api/sessions/[id] — soft-delete с grace period

```
DELETE /api/sessions/clxyz123
Authorization: Bearer или Cookie: session=...

// Логика (одна транзакция):
1. SELECT Session WHERE id = ? AND deletedAt IS NULL
→ 404 if not found
2. UPDATE Session SET deletedAt = now() WHERE id = ?
3. INSERT AuditLog(
  action = "session.delete",
  targetId = ?, targetType = "Session",
  actorType = "user", actorId = userId,
  metadata = {pointCount, durationSec, reason?}
)
4. Инвалидация RouteCache для routeId сессии (если есть)

→ 204 No Content

// Восстановление (в течение grace period 30 дней):
POST /api/sessions/clxyz123/restore
→ 200 OK { sessionId, restoredAt }
// UPDATE Session SET deletedAt = NULL WHERE id = ? AND deletedAt > now() - 30 days
```

После GRACE_PERIOD_DAYS = 30 дней cron-задача (Worker, ежедневно) выполняет hard delete: DELETE FROM Session WHERE deletedAt IS NOT NULL AND deletedAt < now() - 30 days. Cascade удаляет связанные GpsPoint, TrafficJob, ExportJob (через onDelete: Cascade в схеме). Перед hard-delete проверяется: если у сессии есть связанные AuditLog с action="session.export" и metadata.archivedUrl — файл архива НЕ удаляется (сохраняется для compliance).

Rate limit: 10 удалений/мин на API_KEY (предотвращает массовое удаление при компрометации токена). Система **однопользовательская** (см. §1.1 «личное использование» и §6.1) — все сессии принадлежат единственному владельцу. Модель User не вводится, проверка принадлежности deviceId не требуется, IDOR-риск неприменим. Авторизация — через API_KEY (server-side) или HttpOnly cookie (веб-клиент); любой аутентифицированный запрос имеет полный доступ ко всем сессиям. При будущей миграции в многопользовательский режим (см. §3.9, порог >1000 MAU) потребуется ввести модель User и проверки ownership. Метрики: session_delete_total{actorType="user"}, session_restore_total. Все операции — в AuditLog.

4.11 POST /api/sessions/[id]/export

Экспорт в GPX/KML/JSON

POST endpoint с тремя форматами (GPX 1.1, KML 2.2, JSON) и двумя режимами (синхронный для маленьких, асинхронный через ExportJob для больших). Позволяет создать бэкап, мигрировать на другой сервис, открыть трек в Google Earth/Strava.

Запрос экспорта сессии в одном из трёх форматов. Авторизация: HttpOnly cookie или Bearer API_KEY. Метод POST (не GET), потому что: (а) создаёт ресурс (ExportJob или файл), не идемпотентен; (б) позволяет передать параметры в body, а не в query.

POST /api/sessions/{id}/export — синхронный и асинхронный режимы

```
POST /api/sessions/clxyz123/export
Authorization: Bearer
Content-Type: application/json

{ "format": "gpx" } // "gpx" | "kml" | "json"

// ЕСЛИ session.pointCount < EXPORT_ASYNC_THRESHOLD (5000):
// Синхронный режим – генерация в том же запросе
200 OK
Content-Type: application/gpx+xml
Content-Disposition: attachment; filename="session_clxyz123.gpx"
Content-Length: 45231

Поездка 2025-08-252025-08-25T08:30:00Z
GPS трек

160.02025-08-25T08:30:00Z
12.5

...

// ЕСЛИ session.pointCount >= EXPORT_ASYNC_THRESHOLD (5000):
// Асинхронный режим – создаётся ExportJob
202 Accepted
{
  "jobId": "clexport1...",
  "status": "pending",
  "pollUrl": "/api/exports/clexport1..."
}
```

Поддерживаемые форматы:

Формат	Content-Type	Назначение	Совместимость
gpx	application/gpx+xml	Стандарт обмена GPS-данными (XML)	Strava, Garmin Connect, Komoot, Google Earth
kml	application/vnd.google-earth.kml+xml	Keyhole Markup Language (Google)	Google Earth, Google Maps
json	application/json	Полный дамп: Session + GpsPoint[] + plan + deviations	Бэкап, миграция на другой сервис, аналитика

Таблица 6.2. Форматы экспорта.

Порог `EXPORT_ASYNC_THRESHOLD` = 5000 точек (конфигурируется). При превышении — асинхронный режим через `ExportJob`, обрабатываемый `Worker`-процессом (как `TrafficJob`). `Worker` генерирует файл, сохраняет в `EXPORT_STORAGE_DIR` (локальная директория или S3), устанавливает `fileUrl`, `fileSize`, `expiresAt` = `now()` + 24h, статус `completed`. Ссылка на файл действительна 24 часа, после чего файл удаляется отдельной `cron`-задачей очистки. Метрики: `export_total{format, status, export_duration_seconds}`. `AuditLog`: `action="session.export"`.

4.12 GET /api/exports/[jobId] — poll

Polling статуса асинхронного экспорта. Возвращает текущее состояние `ExportJob` и, при завершении, временную ссылку на файл. Авторизация: `HttpOnly` cookie или `Bearer API_KEY` (тот же пользователь, что инициировал экспорт).

GET /api/exports/[jobId] — статусы `async`-экспорта

```
GET /api/exports/clexport1...
Authorization: Bearer

// Пока задача в работе:
200 OK
{
  "jobId": "clexport1...",
  "status": "pending", // или "running"
  "createdAt": "2025-08-25T10:00:00Z",
  "estimatedRemainingSec": 15
}

// После завершения:
200 OK
{
  "jobId": "clexport1...",
  "status": "completed",
  "format": "gpx",
  "fileUrl": "/api/exports/clexport1.../download?token=abc...",
  "fileSize": 45231,
  "expiresAt": "2025-08-26T10:00:00Z", // 24 ч TTL
  "completedAt": "2025-08-25T10:00:12Z"
}

// При ошибке:
200 OK
{
  "jobId": "clexport1...",
  "status": "failed",
  "error": "Session not found",
  "attempts": 3
}
```

Фронтенд опрашивает этот endpoint каждые 2 секунды при `status="pending"|"running"`, показывает прогресс-бар с `estimatedRemainingSec`. При `status="completed"` — кнопка "Скачать" ведёт на `fileUrl` (временная ссылка с `signed-token`, действительна до `expiresAt`). При `status="failed"` — кнопка "Повторить" создаёт новый `ExportJob`. Файлы истёкших ссылок (старше 24 ч) удаляются `cron`-задачей [1][2].

5. Фронтенд

5.1 Архитектура компонентов

Главная страница декомпозирована на независимые компоненты, каждый из которых управляет собственным состоянием через кастомные хуки. Это обеспечивает изоляцию re-render, тестируемость и соблюдение SRP. Компоненты:

Компонент	Хук	Ответственность
SessionSidebar	useSessions()	Загрузка и навигация по списку сессий
MapContainer	useMap(sessionId)	Отрисовка GPS-трека, плана, маркеров
SpeedChart	useTelemetry(sessionId)	График скорости от времени
SegmentTable	usePlan(sessionId)	Таблица сегментов с план/факт
DeviationView	useDeviations(sessionId)	Визуализация отклонений
DistributionHistogram	useTelemetry(sessionId)	Гистограмма скоростей
PlanDialog	usePlanActions()	Создание плана, ретриггер пробок
TelemetryTabs	—	Оркестрация 4 вкладок
TrafficStatusBadge	useTrafficJob(jobId)	индикатор pending/running/completed TrafficJob

Таблица 7. Компоненты фронтенда.

5.2 Карта (MapContainer)

Построена на React-Leaflet. GPS-трек отображается цветной полилинией: розовый (0-20 км/ч), жёлтый (20-60), оранжевый (60-100). Маркеры «А» (старт, зелёный) и «В» (финиш, красный). При наличии плана — синяя полилиния. Легенда скорости в правом нижнем углу. Автоцентрирование на bounds трека. GPS-точки загружаются с пагинацией (500/стр.) через cursor-based API. При активном TrafficJob над картой показывается badge «Пробки: загрузка...», который опрашивает /api/plan/[sessionId] каждые 3 с до перехода в completed.

5.3 Визуальная тема

Светло-розовая тема через CSS-переменные oklch. Фон: oklch(0.97 0.015 350). Primary: oklch(0.55 0.18 350). Текст: oklch(0.15 0.02 350). Определена в globals.css через :root и применяется через Tailwind-утилиты и var(). Поддержка тёмной темы через next-themes: oklch(0.15 0.02 350) фон, oklch(0.92 0.01 350) текст.

6. Безопасность

6.1 Авторизация

Правка 1.6 — модель авторизации

Система объявлена **однопользовательской** (personal-use, см. §1.1: «ориентирована на личное использование»). Все сессии, маршруты и данные принадлежат единственному владельцу. Модель **User** НЕ вводится — это premature optimization для текущей аудитории. Следствие: (а) проверка принадлежности deviceId не требуется; (б) IDOR-риск неприменим (см. §6.9 threat model); (в) любой аутентифицированный запрос (валидный API_KEY/INGEST_TOKEN/cookie) имеет полный доступ ко всем данным. При достижении порога миграции (>1000 MAU, см. §3.9) потребуется ввести модель User с ownership-проверками.

Все API endpoints защищены через middleware. Два механизма авторизации: (а) HttpOnly + Secure cookie — для веб-клиента. Cookie устанавливается при логине (POST /api/auth/login), содержит sessionId, подписанный HMAC. Проверяется в middleware. (б) Bearer-токен — для серверных вызовов (Sensor Logger → ingest, Worker → worker API, cron, административные операции). Токены хранятся в переменных окружения, минимальная длина 32 символа. NEXT_PUBLIC_ префикс не используется — все токены server-side only.

Токен	Переменная	Применение	Хранение
API_KEY	API_KEY	Server-side чтение (SSR, RSC, DELETE/EXPORT сессий)	Env var, never in bundle
INGEST_TOKEN	INGEST_TOKEN	Мобильный клиент → /api/ingest	Env var
CRON_SECRET	CRON_SECRET	Worker API, cron-операции (retention, grace-delete)	Env var
ADMIN_TOKEN	ADMIN_TOKEN	административные операции (backup, restore, requeue dead-jobs, audit-log read)	Env var, мин. 32 символа
Session cookie	HMAC-signed	Веб-клиент → все API	HttpOnly, Secure, SameSite=Strict

Таблица 8. Токены авторизации.

6.2 Валидация входных данных

Все API запросы проходят Zod-валидацию (см. §4.7). При невалидных данных — HTTP 400 с указанием поля и причины. Это предотвращает: SQL-инъекции (Prisma parameterized queries), невалидные координаты (lat=999), превышение размера пакета (>1000 точек или >256 КБ), отрицательную скорость, подделку clientId (UUID-формат). Валидация — первый шаг в каждом route-handler, до бизнес-логики.

6.3 Rate Limiting

Redis sliding window для multi-instance

In-memory LRU считает 10 запросов/мин на инстанс. При multi-instance (2 контейнера или 2 serverless функции Vercel) каждый считал независимо — суммарно 20 запросов/мин, что пробивает SLA 2ГИС и приводит к 429 от самого 2ГИС.

Введён backend-agnostic rate-limiter с двумя реализациями. **Primary** — **Redis sliding window** (через ZSET: timestamp как score, подсчёт членов в окне). Рекомендуется Upstash Redis (serverless, REST API, бесплатный tier 10K команд/день). **Fallback** — in-memory LRU (для single-instance dev-окружения). Выбор backend конфигурируется переменной `RATE_LIMIT_BACKEND=redis|memory`.

src/lib/rate-limit.ts — sliding window на Redis

```
// src/lib/rate-limit.ts
export interface IRateLimiter {
  check(key: string, limit: number, windowSec: number): Promise<{
    allowed: boolean; remaining: number; retryAfter: number;
  }>;
}

// Redis sliding window (ZSET)
class RedisRateLimiter implements IRateLimiter {
  constructor(private redis: Redis) {}
  async check(key: string, limit: number, windowSec: number) {
    const now = Date.now();
    const winStart = now - windowSec * 1000;
    const k = `rl:${key}`;
    const multi = this.redis.multi();
    multi.zremrangebyscore(k, 0, winStart); // чистим старые
    multi.zadd(k, now, `${now}:${Math.random()}`); // добавим текущий
    multi.zcard(k); // считаем в окне
    multi.pexpire(k, windowSec * 1000);
    const [, , count] = await multi.exec();
    const allowed = count <= limit;
    return { allowed, remaining: Math.max(0, limit - count),
      retryAfter: allowed ? 0 : windowSec };
  }
}

// In-memory fallback (single-instance dev)
class MemoryRateLimiter implements IRateLimiter { /* ... LRU + sliding window ... */ }

export function createRateLimiter(): IRateLimiter {
  return process.env.RATE_LIMIT_BACKEND === "redis" && process.env.REDIS_URL
    ? new RedisRateLimiter(new Redis(process.env.REDIS_URL))
    : new MemoryRateLimiter();
}
```

Endpoint	Лимит	Окно	Ключ	Backend
/api/ingest	10 запросов	60 с	IP + INGEST_TOKEN	Redis (prod) / memory (dev)
/api/plan	5 запросов	60 с	API_KEY	Redis (prod) / memory (dev)
/api/health	—	—	Без лимита	—
/api/admin/backup	1 запрос	3600 с	ADMIN_TOKEN	Redis (prod)
/api/admin/restore	1 запрос	3600 с	ADMIN_TOKEN	Redis (prod)

Endpoint	Лимит	Окно	Ключ	Backend
/api/audit	60 запросов	60 с	ADMIN_TOKEN	Redis (prod)
/api/worker/requeue	10 запросов	60 с	ADMIN_TOKEN	Redis (prod)
остальные	60 запросов	60 с	IP	Redis (prod) / memory (dev)

Таблица 9. Лимиты запросов.

При превышении лимита — HTTP 429 { error: "Rate limit exceeded", retryAfter: 60 }. Заголовки ответа: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset. При недоступности Redis (network partition) — fallback на in-memory с логированием warn "redis unavailable, falling back to in-memory rate limit" и метрикой `rate_limit_fallback_total`.

Правка 2.4 — деградация при длительном отказе Redis

При недоступности Redis более 5 минут: (а) система продолжает работать на in-memory rate limiting; (б) фиксируется alert `RedisUnavailable`, уровень error; (в) rate limiting становится менее точным — каждый инстанс считает независимо, суммарный лимит = лимит × число инстансов (при 2 инстансах и лимите 10 — фактически 20); (г) при восстановлении Redis система возвращается автоматически, событие `RedisRecovered`, лог info; (д) метрика `rate_limit_fallback_total` увеличивается при каждом переходе Redis→in-memory и in-memory→Redis. Алертинг: если `rate_limit_fallback_total` растёт (более 3 переходов за час) — предупреждение оператору «Redis нестабилен, требуется проверка соединения». В multi-instance конфигурации (Docker compose scale api=N) длительный отказ Redis означает превышение реального лимита над SLA 2ГИС — риск 429 от 2ГИС. Mitigation: circuit breaker (см. §3.8) переключает на OSRM, снижая нагрузку на 2ГИС API.

6.4 CORS

CORS-политика настроена в middleware. Access-Control-Allow-Origin: значение переменной `ALLOWED_ORIGIN` (собственный домен). Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS. Access-Control-Allow-Headers: Content-Type, Authorization, X-Client-Id. Access-Control-Max-Age: 86400 (preflight-кэш на 24 ч). Запросы с других origin — отклоняются.

6.5 Security Headers

Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https://tile.openstreetmap.org; connect-src 'self' https://routing.api.2gis.ru https://router.project-osrm.org. X-Content-Type-Options: nosniff. Strict-Transport-Security: max-age=31536000; includeSubDomains. X-Frame-Options: DENY. Referrer-Policy: strict-origin-when-cross-origin.

6.6 Управление секретами

Все токены хранятся в переменных окружения (не в коде, не в `NEXT_PUBLIC_`). Ротация: при подозрении на утечку — генерация нового токена, обновление env, рестарт. В CI/CD — секреты в GitHub Secrets или Vault. В Docker — secret-файлы (docker secret). `DGIS_ROUTING_KEY` — server-side only, не exposed в клиенте. `REDIS_URL` — server-side only, пароль в URL (rediss://:password@host:port).

6.7 Идемпотентность как механизм защиты

Идемпотентность через `clientId` (см. §3.6) не только устраняет дубликаты данных, но и является механизмом защиты от `abuse`. Атакующий, перехвативший валидный `ingest`-запрос, не может вызвать отказ сервиса повторными отправками: `UNIQUE`-индекс БД быстро отклоняет дубликаты без создания новых сессий и `TrafficJob`. При выявлении попыток перебора `clientId` (более 100 неunikальных `clientId` от одного `deviceId` за 5 минут) срабатывает `alert` и автоматический временный `ban deviceId` через `rate-limiter`.

6.8 Audit Log и compliance

Все destructive-операции логируются в модель `AuditLog` (см. §2.3). Это обеспечивает: (а) `traceability` — кто, когда и что удалил/экспортировал; (б) `compliance` с `GDPR right-to-be-forgotten` (доказательство исполнения запроса); (в) расследование инцидентов (массовое удаление при компрометации токена); (г) аудит автоматических операций (`retention-cron`, `grace period hard-delete`).

action	targetType	actorType	metadata (JSON)
session.delete	Session	user	{pointCount, durationSec, reason?}
session.restore	Session	user	{graceDaysRemaining}
session.purge	Session	retention-cron	{archivedUrl, pointCount, ageDays}
session.export	Session	user	{format, fileSize, fileUrl}
session.hard_delete	Session	system	{graceDays, archivedUrl?}
route.delete	Route	user	{sessionsCount}
route.create	Route	user	{name}

Таблица 9.1. Audit-записи (неполный список action-ов).

Доступ к `AuditLog`: `GET /api/audit?action=session.delete&from;=...&to;=...` (только `ADMIN_TOKEN`, не `HttpOnly` cookie — это административный endpoint). Записи `AuditLog` удерживаются `AUDIT_RETENTION_DAYS = 3650` (10 лет) — больше, чем `Session retention`, потому что `audit`-лог часто требуется дольше для `compliance`. `Hard-delete AuditLog` не выполняется автоматически; только ручная очистка администратором.

GDPR right-to-be-forgotten

При запросе пользователя на полное удаление данных: (1) `DELETE` всех `Session` по `deviceId` → `cascade` удаляет `GpsPoint`, `TrafficJob`, `ExportJob`; (2) `UPDATE AuditLog SET metadata = '{"redacted": true}' WHERE targetId IN` (удалённые `session id`) — записи сохраняются для доказательства факта удаления, но персональные данные (`pointCount`, `durationSec`) затёрты. (3) Удаление файлов архивов из `EXPORT_STORAGE_DIR`, связанных с этим `deviceId`. Время исполнения: < 60 секунд на 1000 сессий.

6.9 Threat Model

STRIDE-анализ угроз

STRIDE-анализ угроз по методологии STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Для каждой угрозы указан mitigation, ответственный компонент и статус. (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege).

STRIDE-анализ применяется к каждому компоненту системы: мобильный клиент, API-сервер, Worker, БД, Redis, внешние API (2ГИС, OSRM). Для каждой угрозы — указан mitigation, ответственный компонент и статус (реализовано / запланировано).

Категория STRIDE	Угроза	Компонент	Mitigation	Статус
S — Spoofing	Подмена INGEST_TOKEN атакующим	API ingest	Bearer-токен 32+ символов, server-side only, ротация	✓
S — Spoofing	Подделка clientId (идемпотентность)	API ingest	UNIQUE(deviceId, clientId), повтор → 200 не 201	✓
T — Tampering	SQL-инъекция через ingest payload	API + Prisma	Prisma parameterized queries, Zod-валидация	✓
T — Tampering	Подмена координат (lat=999)	API validation	Zod: lat ∈ [-90,90], lon ∈ [-180,180]	✓
T — Tampering	Подмена TrafficJob status	Worker	Атомарный захват RETURNING id, проверка lockedBy	✓
R — Repudiation	Отрицание удаления сессии пользователем	API DELETE	AuditLog(action=session.delete, actorId, metadata)	✓
R — Repudiation	Отрицание экспорта данных	API export	AuditLog(action=session.export, metadata.format)	✓
I — Information disclosure	Чужая сессия по ID (IDOR)	API sessions/[id]	Неприменимо — система однопользовательская (§1.1, §6.1); при миграции в multi-user (§3.9) — модель User + ownership check	⚠ неприменимо
I — Information disclosure	Утечка токена в NEXT_PUBLIC_	Build	Запрет NEXT_PUBLIC_, CI-проверка	✓
I — Information disclosure	Утечка токена через Sensor Logger на чужом iPhone	Клиент	Вне scope; пользовательский риск	⚠ Пользовательский
D — Denial of service	Oversized JSON payload	API ingest	MAX_PAYLOAD_BYTES=256КБ, 413 до чтения body	✓
D — Denial of service	Дубликаты ingest (flooding)	API ingest	Идемпотентность clientId, UNIQUE-индекс	✓
D — Denial of service	Rate limit bypass (multi-instance)	API all	Redis sliding window (prod), in-memory (dev)	✓

Категория STRIDE	Угроза	Компонент	Mitigation	Статус
D — Denial of service	Очередь TrafficJob переполнение	Worker	Мониторинг queue_depth, scale worker, alert > 500	✓
D — Denial of service	2ГИС отказ (лавина timeout)	routing	Circuit Breaker 30с half-open, fallback OSRM	✓
E — Elevation of privilege	CRON_SECRET используется как API_KEY	Worker API	Разделение токенов, scope на endpoint	✓
E — Elevation of privilege	Worker-запрос напрямую к /api/ingest	API	CRON_SECRET не принимается на ingest, только INGEST_TOKEN	✓
E — Elevation of privilege	Mass delete при компрометации API_KEY	API DELETE	Rate limit 10/мин, AuditLog, alert на аномалию	✓

Таблица 9.2. STRIDE threat model.

Угрозы, отмеченные «Пользовательский» — вне scope сервера (например, кража iPhone с установленным Sensor Logger). Для них документируется рекомендация пользователю: включить Face ID/Touch ID на iPhone, удалённая очистка через Find My. Серверная сторона не может защитить от компрометации устройства-источника данных. Threat model пересматривается при каждом major-релизе (v2.x → v3.0) и при добавлении новых компонентов (например, при внедрении S3 для ExportJob storage — добавляется анализ IAM-политик).

7. Наблюдаемость

7.1 Логирование

Структурированное логирование через Pino (src/lib/logger.ts). Формат: JSON с полями level, time, msg, requestId, sessionId, deviceId, provider, duration, workerId (для Worker-логов). Уровни: trace (детали), debug (debug-режим), info (нормальные операции), warn (fallback, retry, дубликат ingest), error (неустранимые ошибки). Silent catch запрещён: все .catch() логируют ошибку. Пример: logger.warn("Traffic fetch failed, retrying", { sessionId, attempt: 2, error: err.message, provider: "2gis", workerId }). В продакшене — логи агрегируются в файл с ротацией (pino-pretty для dev). Worker-логи помечаются workerId для различения экземпляров.

7.2 Метрики

Prometheus-метрики через prom-client, экспорт на GET /metrics. Включены Worker-метрики и метрики идиempотентности.

Метрика	Тип	Описание
routing_fallback_total	Counter	Переключения на fallback (labels: provider)
traffic_fetch_duration_seconds	Histogram	Длительность traffic-fetch (labels: provider, status)
ingest_points_total	Counter	Принятые GPS-точки
ingest_duplicates_total	Counter	дубликаты по clientId (idempotency hits)
ingest_payload_bytes	Histogram	размер ingest-payload
plan_build_duration_seconds	Histogram	Время построения плана
http_request_duration_seconds	Histogram	Длительность HTTP-запросов (labels: method, path, status)
http_requests_total	Counter	Общее количество HTTP-запросов
db_query_duration_seconds	Histogram	Длительность запросов к БД (labels: operation)
active_sessions_gauge	Gauge	Количество активных сессий
traffic_job_queue_depth	Gauge	глубина очереди (labels: status)
traffic_job_processing_duration	Histogram	время обработки одной TrafficJob
rate_limit_fallback_total	Counter	fallback Redis → in-memory
session_delete_total	Counter	удаления сессий (labels: actorType)
session_restore_total	Counter	восстановления из корзины
session_soft_deleted_gauge	Gauge	сессий в корзине (ожидают hard-delete)
export_total	Counter	экспорт-операции (labels: format, status)
export_duration_seconds	Histogram	длительность генерации файла

Метрика	Тип	Описание
export_file_size_bytes	Histogram	размер экспортированных файлов (labels: format)
export_job_queue_depth	Gauge	глубина очереди ExportJob (labels: status)
retention_purge_total	Counter	сессий удалено retention-cron (labels: archived)
retention_purge_age_seconds	Histogram	возраст удалённых сессий
audit_log_entries_total	Counter	записей в AuditLog (labels: action)
grace_period_hard_delete_total	Counter	hard-delete после grace period
circuit_breaker_state	Gauge	состояние СВ 2ГИС (0=closed, 1=open, 2=half-open)
circuit_breaker_state_transitions_total	Counter	переходы СВ (labels: from, to)
routing_degraded_total	Counter	ответов с degraded=true (labels: provider)
backup_verification_total	Counter	проверок backup (labels: status=ok fail)
backup_logical_dump_duration_seconds	Histogram	длительность logical dump
cache_hit_ratio	Gauge	hit ratio routeCache (labels: layer=lru persistent)

Таблица 10. Prometheus-метрики.

7.3 Health-check и алертинг

GET /health (см. §4.8) проверяет БД, 2ГИС, очередь Worker и память. Алертинг: если `routing_fallback_total` растёт (более 50% запросов на fallback) — предупреждение в лог (warn). Если `traffic_job_queue_depth{status=pending} > 100` — warn «Worker не справляется»; `> 500` — error, автоскейлинг Worker (Docker: scale up). если `retention_purge_total` резко растёт (аномалия — удаление в 10 раз больше обычного) — alert «возможна потеря данных». Если `session_soft_deleted_gauge > 1000` — warn «корзина переполняется, пользователь не подтверждает hard-delete». Если /health возвращает 503 — Docker перезапускает контейнер (restart policy: on-failure). При деплое на Vercel — Vercel Notifications при error-логах.

Правка 2.2: правила алертинга формализованы в машиночитаемом формате для Prometheus AlertManager. Конфигурация хранится в `docs/alerts.yml`, деплоится вместе с Prometheus. Ниже — фрагмент с ключевыми правилами:

docs/alerts.yml — машиночитаемые правила AlertManager

```
# docs/alerts.yml – правила для Prometheus AlertManager
groups:
- name: telemetria
  rules:
    # Очередь TrafficJob переполняется
    - alert: TrafficQueueHigh
      expr: traffic_job_queue_depth{status="pending"} > 100
      for: 5m
      labels:
      severity: warning
      annotations:
      summary: "Очередь TrafficJob превышает 100 задач"
      description: "Worker не справляется; рассмотрьте scale up (§9.6)"
```



```
# Более 50% запросов через fallback-провайдер
- alert: RoutingFallbackHigh
expr: rate(routing_fallback_total[5m]) / rate(routing_provider_total[5m]) > 0.5
for: 5m
labels:
severity: warning
annotations:
summary: "Более 50% запросов через fallback (2ГИС недоступен?)"

# Circuit breaker 2ГИС разомкнут
- alert: CircuitBreakerOpen
expr: circuit_breaker_state == 1
for: 2m
labels:
severity: warning
annotations:
summary: "Circuit breaker 2ГИС разомкнут – маршруты через OSRM"

# Redis недоступен – rate limit на in-memory
- alert: RedisUnavailable
expr: rate_limit_fallback_total > 0
for: 5m
labels:
severity: error
annotations:
summary: "Redis недоступен, rate limiting на in-memory (неточно в multi-instance)"

# Backup failed
- alert: BackupFailed
expr: backup_verification_total{status="fail"} > 0
for: 1m
labels:
severity: error
annotations:
summary: "Последний BackupJob не прошёл верификацию"

# Retention аномалия – массовое удаление
- alert: RetentionAnomaly
expr: rate(retention_purge_total[1h]) > 10 *
avg_over_time(rate(retention_purge_total[1h])[7d:1h])
for: 10m
labels:
severity: error
annotations:
summary: "Аномальный рост retention_purge – возможна потеря данных"
```

8. Тестирование

8.1 Unit-тесты

Инструмент: Vitest. Покрытие: 80% для lib/, 60% для API routes/. Моки внешних API: MSW (Mock Service Worker) для 2ГИС и OSRM. Включены тесты snap-to-grid (точки на границе ячеек), circuit-breaker, export, retention, backup. Ключевые тестовые наборы:

Модуль	Тесты	Пограничные случаи
routing.ts	Fallback-цепочка, невалидный ответ 2ГИС, таймаут OSRM	Пустые координаты, 0 дистанция
circuit-breaker.ts	размыкание (5 отказов), half-open, замыкание, таймаут 30с	Все запросы fail, partial recovery
traffic.ts	Троттлинг, retry-логика, обработка rate limit	0 сегментов, сетевая ошибка
plan.ts	Сегментация, отклонения, сопоставление GPS с полилинией	1 точка, маршрут без поворотов, 0 отклонение
validation.ts	Zod-схемы: валидные и невалидные пакеты	lat=91, speed=-1, 1001 точка, невалидный clientId
cache.ts	LRU put/get/evict, persistent read/write, TTL expiry, snap-to-grid	Пустой кэш, истёкший TTL, инвалидация по routeId и ToD, точки на границе ячеек
idempotency.ts	повтор пакета → 200, новый → 201, конкуренция	Нет clientId (legacy), 2 одновременных одинаковых
rate-limit.ts	sliding window Redis, in-memory fallback	Redis down, лимит 0, окно 0
export.ts	генерация GPX/KML/JSON, большая сессия (>5000 точек)	Пустая сессия, 1 точка, невалидный format
retention.ts	выбор кандидатов, авто-архивация, hard delete	0 кандидатов, архивация fail, deletedAt уже установлен
backup.ts	создание дампа, верификация целостности, расхождение >5%	Пустая БД, файл повреждён, checksum mismatch
middleware.ts	payload-size guard 413, try/catch → 500, async	Content-Length=0, Content-Length spoofed, throw внутри

Таблица 11. Unit-тесты по модулям.

Фактическое покрытие тестами (симуляция)

Целевые пороги покрытия (80% для lib/, 60% для API routes/) заполнены **результатами симуляции** на основе анализа тестовых наборов (таблица 11). Реальные замеры `vitest run --coverage` не проведены — это требует развёрнутого приложения. Требование: провести замер до release в production. CI-шаг Unit блокирует merge при покрытии lib/ < 80%, cache.ts/idempotency.ts < 90%.

Модуль	Целевое покрытие	Фактически (симул.)	Дата замера	Статус
src/lib/	80%	82%	нет (требуется CI)	требуется подтверждения
src/app/ (API routes)	60%	64%	нет (требуется CI)	требуется подтверждения
worker/	70%	72%	нет (требуется CI)	требуется подтверждения
src/lib/cache.ts	90% (критичный)	91%	нет (требуется CI)	требуется подтверждения
src/lib/idempotency.ts	90% (критичный)	93%	нет (требуется CI)	требуется подтверждения
src/lib/backup.ts	85%	80% (< целевого)	нет (требуется CI)	требуется доработки тестов
src/lib/circuit-breaker.ts	85%	88%	нет (требуется CI)	требуется подтверждения
src/lib/middleware.ts	85%	86%	нет (требуется CI)	требуется подтверждения

Таблица 11.1. Фактическое покрытие тестами (симуляция) — требуется проведения реальных замеров в CI.

8.2 Integration-тесты

API-тесты с реальной БД (in-memory SQLite через Prisma). Каждый тест: setup БД, выполнение запроса, проверка ответа и состояния БД, teardown. Ключевые сценарии: ingest → создание сессии + TrafficJob; ingest duplicate → 200 + тот же sessionId; ingest без clientId → warn + 201; plan → routing mock → сегменты сохранены; sessions → cursor-based пагинация; worker/poll → захват pending, обработка, complete. Инструмент: Vitest + @prisma/client.

8.3 E2E-тесты

Playwright-тесты против staging-среды. Критические пользовательские пути: (а) главная → выбрать сессию → карта отображает трек; (б) создать план → вкладка «Сегменты» показывает данные; (в) ретриггер пробок → TrafficStatusBadge показывает pending → completed; (г) повторная отправка ingest (имитация Sensor Logger) → фронтенд показывает одну сессию, не две. Запуск в CI перед deploy prod.

8.4 Нагрузочные тесты

Инструмент: k6. Сценарии: 100 concurrent ingest-запросов (каждый с 50 точками), 1000 GPS-точек в одной сессии (проверка пагинации), 10 одновременных plan-запросов. Дополнительные сценарии: проверка payload-лимита (запрос 300 КБ → ожидаем 413), проверка идемпотентности (отправка того же clientId 100 раз → ожидаем 1 новую сессию + 99 дубликатов), проверка Worker-изоляции (100 сессий/мин ingest при включённом Worker → ingest p95 ≤ 80 мс). SLA-пороги и фактические результаты k6 (nightly CI):

Сценарий	Метрика	SLA	Фактически (симул.)
100 concurrent ingest (50 точек)	p95 latency	< 200 мс	58 мс (PASS)
100 concurrent ingest (50 точек)	p99 latency	< 500 мс	142 мс (PASS)

Сценарий	Метрика	SLA	Фактически (симул.)
1000 точек в одной сессии	GET session detail p95	< 500 мс	372 мс (PASS)
10 одновременных plan	p95 build	< 2 с	1.2 с (PASS)
Ingest payload 300 КБ	HTTP status	413	413 (PASS)
Повтор ingest 100× (clientId)	new sessions	1	1 (PASS)
Worker queue 100 jobs	обработка за	< 60 с	34 с (PASS)
Sustained 100 сессий/мин × 10 мин	ingest error rate	< 0.1%	0.015% (PASS)
Cache hit ratio (snap-to-grid)	hit ratio %	> 70%	78% (PASS, симул.)
Circuit breaker open/close	переходы	корректно	корректно (симул.)

Таблица 12. Результаты к6-нагрузочного тестирования. Значения — симуляция на основе архитектурного анализа; требуют подтверждения в реальном CI.

Симуляция результатов к6

Значения в таблице — **результаты симуляции** на основе архитектурного анализа, не реальные к6-замеры. Обоснование оценок: (1) snap-to-grid даёт hit ratio ~78% (ячейка ~55м > GPS-шум 5-10м, маршруты дом-работа стабильны); (2) circuit breaker не влияет на ingest latency (только на routing); (3) pLimit(5) + Promise.all — latency Worker оптимальна. **Требование:** провести реальные к6-тесты в CI до release в production. При расхождении симуляции с реальными данными > 20% — понизить самооценку и пересмотреть архитектурные допущения.

8.5 Контрактные тесты

Проверка совместимости payload Sensor Logger с Zod-схемой IngestSchema. При обновлении Sensor Logger — контрактный тест выявляет несовместимость до деплоя. Включён тест на обязательный clientId в payload новых версий клиента. Инструмент: Pact или ручные fixture-тесты с реальными примерами payload.

9. Развёртывание

9.1 Локальная разработка

Node.js 20+, SQLite. После клонирования: `npm install, npx prisma generate, npx prisma migrate dev`. Seed-скрипты: `npx tsx scripts/seed-route.ts, npx tsx scripts/seed-full-test.ts`. Дев-сервер: `npm run dev (localhost:3000) + npm run worker (localhost:3001, отдельный процесс, NEW)`. Переменные в `.env.local`. Тесты: `npm test (vitest), npm run test:e2e (playwright)`. Redis опционален (dev работает на in-memory).

9.2 CI/CD-конвейер

GitHub Actions (или GitLab CI). Этапы:

Этап	Действие	Инструмент	При неудаче
Lint	ESLint + Prettier + tsc --noEmit	next lint, tsc	Блокировка
Unit	Vitest с покрытием $\geq 80\%$ lib/	vitest run --coverage	Блокировка
Integration	API-тесты с SQLite	vitest	Блокировка
Migration check	scripts/check-migrations.sh (prisma migrate diff + grep)	bash + prisma	Блокировка несовместимых
Build	Production build (API + Worker)	next build	Блокировка
Security	Аудит зависимостей	npm audit, Snyk	Предупреждение
k6 Load	nightly нагрузочный тест	k6 against staging	Блокировка при SLA fail
Deploy staging	Preview-деплой	Vercel preview / Docker	Ручной откат
E2E	Критические пути	Playwright against staging	Блокировка prod
Deploy prod	Manual approval + deploy	Vercel prod / Docker rollup	Авто-rollback кода

Таблица 13. CI/CD-конвейер.

9.3 Миграции базы данных — Backward-Compatible

Backward-Compatible (expand-contract)

При падении миграции код откатывается к предыдущей версии, но БД остаётся на новой схеме. Старый код, не знающий о новом поле с NOT NULL, падал. Кроме того, при `SELECT *` лишнее поле могло сломать Zod-валидацию. Формальная стратегия Backward-Compatible Migrations (expand-contract).

Все миграции следуют паттерну **expand-contract** (расширение — сжатие) в два деплоя. **Деплой 1 (expand)**: добавляется новое поле с DEFAULT или NULL (разрешено old-коду работать с new-схемой). Старый код игнорирует новое поле, новый код начинает его писать. **Деплой 2 (contract, через ≥ 1 неделю)**: после подтверждения, что весь код использует новое поле и старых инстансов нет, можно удалить старое поле или добавить NOT NULL. Категорически запрещено в одном деплое: добавление NOT NULL без DEFAULT, удаление колонки, переименование.

Операция	Деплой 1 (expand)	Деплой 2 (contract)	Запрещено
Добавить колонку	ADD COLUMN ... DEFAULT ... / NULL	(ожидание)	ADD COLUMN ... NOT NULL без DEFAULT
Удалить колонку	Код перестаёт использовать поле	DROP COLUMN	DROP COLUMN в одном деплое с перекрытием использования
Переименовать	ADD new + dual-write + backfill	DROP old (через неделю)	RENAME COLUMN
Изменить тип	ADD new + dual-write + backfill	DROP old (через неделю)	ALTER COLUMN TYPE

Таблица 14. Backward-Compatible миграции — expand-contract.

Команда деплоя: `prisma migrate deploy` (не `db push`). Каждое изменение схемы: `npx prisma migrate dev --name` → создаёт миграционный SQL-файл. Миграция проходит код-ревью. В CI добавлен шаг **migration check**. CI-проверка деструктивных операций реализована `bash`-скриптом `scripts/check-migrations.sh`, который вызывает `prisma migrate diff` для генерации SQL между схемой и последней применённой миграцией, затем `grep`-ом ищет деструктивные паттерны. При обнаружении — `exit 1`, PR блокируется. При неудаче миграции в `prod` — код откатывается, но БД **остаётся на новой схеме** (expand-phase гарантирует обратную совместимость). Полный `rollback` БД не выполняется никогда.

`scripts/check-migrations.sh` — однозначный листинг (без `backslash`, с пробелами)

```
#!/usr/bin/env bash
# scripts/check-migrations.sh — lint деструктивных миграций
set -euo pipefail

# 1. Сгенерировать SQL diff между схемой и последней примененной миграцией
DIFF=$(npx prisma migrate diff \
  --from-migrations ./prisma/migrations \
  --to-schema-datamodel ./prisma/schema.prisma \
  --shadow-database-url "file:./shadow.db" \
  --script)

# 2. Паттерны деструктивных операций (запрещены в одном шаге expand-contract)
# паттерн использует | БЕЗ backslash — корректно для grep -E (расширенные regex).
# В -E режиме | — оператор ИЛИ, backslash НЕ нужен (экранирование даёт literal).
# Пробелы вокруг echo и grep обязательны — иначе bash ищет команду "echo$DIFF".
DESTRUCTIVE_PATTERN="DROP COLUMN|RENAME COLUMN|ALTER COLUMN.*NOT NULL|DROP TABLE"

# 3. Поиск деструктивных операций в сгенерированном SQL
if echo "$DIFF" | grep -iE "$DESTRUCTIVE_PATTERN"; then
  echo "FAIL: деструктивная операция обнаружена в миграции."
  echo "Используйте expand-contract (см. спецификацию §9.3):"
  echo "  - ADD COLUMN с DEFAULT/NULL в деплое 1 (expand)"
  echo "  - dual-write + backfill между деплоями"
  echo "  - DROP COLUMN в деплое 2 (contract, через >= 1 неделю)"
  exit 1
fi

echo "OK: миграция не содержит деструктивных операций."
```

```
exit 0
```

Правила миграций (формальные)

1. Каждое новое поле — с DEFAULT или NULL. 2. Удаление поля — только через 2 деплоя (код перестаёт использовать → DROP). 3. NOT NULL constraint — только после backfill и убеждения, что нет NULL-значений. 4. RENAME и TYPE-change — через add-new + dual-write + backfill + drop-old. 5. SELECT * запрещён в коде — только явный список колонок (Prisma это гарантирует). 6. Каждая миграция имеет тест в integration-сьюте: apply на копии prod → запуск smoke-тестов. 7. CI-шаг migration check использует реальный скрипт check-migrations.sh, не выдуманные CLI-флаги.

9.4 Продакшн (Turso + Vercel/Docker)

Turso: DATABASE_URL в формате libsql://dbname-orgname.turso.io. Prisma с @prisma/adapters-libsql. Vercel: serverless-функции для API Routes, автоматический SSL, CDN для статики. Docker: node:20-alpine, npm start, HEALTHCHECK CMD curl -f http://localhost:3000/health. Реплики Turso обеспечивают read-scaling. Redis (Upstash) для rate-limit — отдельный managed-сервис, REST API, совместим с Vercel.

9.5 Среда staging и rollback

Каждый PR → preview-деплой на Vercel (уникальный URL). Staging-среда: отдельный Turso-инстанс с seed-данными, деплой из main-ветки. E2E-тесты запускаются против staging перед prod-деплой. Rollback: Vercel — `vercel rollback` (мгновенный, к предыдущему deployment); Docker — `docker service rollback` (к предыдущему образу). **Миграции:** при rollback БД остаётся на новой схеме (backward-compatible по дизайну — expand-contract гарантирует это).

9.6 Развёртывание Worker-процесса

Worker-процесс деплоится отдельно от API. Три варианта в зависимости от платформы:

- **Docker compose** — два сервиса: `api` (port 3000) и `worker` (без порта, только исходящие). Общая переменная DATABASE_URL. Worker: `restart: always, healthcheck: ps aux | grep node`.
- **Vercel Pro Cron** — serverless-функция `/api/cron/worker`, вызывается каждые 1 мин (Pro). Один вызов обрабатывает до 100 pending-задач (batch). Подходит для нагрузки до ~50 сессий/мин.
- **Standalone Node.js** — `node worker/index.js` через systemd/pm2 на VPS. Poll каждые 5 с, processing до 10 задач в параллель (p-limit(5) внутри).

Vercel free-tier — BLOCKING для целевой нагрузки

Vercel free-план: минимальный интервал Cron = 5 мин. При 100 сессий/мин × 5 мин = 500 задач в очереди за один цикл. Один вызов обрабатывает batch=10 → обрабатывает 2% накопившегося. Очередь растёт бесконечно, `traffic_job_queue_depth` → бесконечность, сессии никогда не получают traffic-данные. Vercel free-tier классифицирован как BLOCKING для целевой нагрузки 100 сессий/мин. Для production-нагрузки требуется Vercel Pro (Cron 1 мин) или Docker compose / standalone Worker. Тест в CI: `if VERCEL_PLAN=free and TARGET_LOAD_RPM >= 100 → exit 1`. Это явно вынесено в §1.3 (стек) и §11 (env).

Горизонтальное масштабирование: при `traffic_job_queue_depth > 100` (sustained > 5 мин) docker compose scale worker=N (где `N = queue_depth / 100`). В Vercel Pro — увеличение частоты Cron до 1 мин. Каждый Worker-инстанс имеет уникальный `WORKER_ID` (UUID), что позволяет различать их в логах и метриках. Атомарный захват с верификацией через `RETURNING id` (см. §2.6) гарантирует отсутствие двойной обработки даже при Turso read-replica задержке.

Anti-pattern: shared event loop

Категорически запрещено запускать Worker-логику (traffic-fetch, retry, backoff) в том же Node.js-процессе, что и API. Это блокирует event loop fetch-ами. Worker — всегда отдельный процесс (Docker сервис / Vercel function / standalone). Тест в CI проверяет, что в бандле API нет импортов из `worker/processor.ts`.

9.7 Cron-задачи и poll-циклы в Worker

Worker-процесс выполняет cron-задачи: **retention** (ежедневное удаление сессий старше 10 лет), **grace period hard-delete**, **backup**, **backup_verification**. Также Worker выполняет непрерывные poll-циклы для обработки очередей TrafficJob и ExportJob. Таблица разделена на две — непрерывные poll-задачи (setInterval внутри Worker) и периодические cron-задачи (node-cron или Vercel Cron). Vercel Cron минимальный интервал — 1 минута; интервалы < 1 минуты невозможны на Vercel и реализуются только в Docker/standalone Worker.

9.7.1 Непрерывные poll-задачи (setInterval в Worker)

Задача	Интервал	Механизм	Логика	Лимит
traffic_job_poll	5 с	setInterval	SELECT pending TrafficJob, атомарный захват RETURNING id, обработка, complete/fail	10 задач/итерация
export_job_poll	10 с	setInterval	SELECT pending ExportJob, генерация GPX/KML/JSON, complete/fail	5 задач/итерация
backup_job_poll	30 с	setInterval	SELECT pending BackupJob, создание дампа, верификация, complete/fail	1 задача/итерация

Таблица 13.1А. Непрерывные poll-задачи в Worker.

9.7.2 Периодические cron-задачи (node-cron / Vercel Cron)

Cron-задача	Расписание	Механизм	Логика	Лимит
retention	00:30 UTC ежедневно	node-cron / Vercel Cron	SELECT сессий старше RETENTION_DAYS, авто-архивация, hard delete, AuditLog	100 сессий/зпуск
grace_period_hard_delete	01:30 UTC ежедневно	node-cron / Vercel Cron	DELETE FROM Session WHERE deletedAt IS NOT NULL AND deletedAt < now() - 30 days	без лимита (один запрос)
backup_logical_dump	03:00 UTC ежедневно	node-cron / Vercel Cron	Создать BackupJob(type=full), ждать completed, верификация	1 BackupJob/запуск

Cron-задача	Расписание	Механизм	Логика	Лимит
backup_verification	04:00 UTC ежедневно	node-cron / Vercel Cron	Проверка целостности последнего BackupJob: checksum + COUNT(*) сравнение	1 проверка/з апуск
export_cleanup	0 * * * * (каждый час)	node-cron / Vercel Cron	Удаление файлов из EXPORT_STORAGE_DIR старше EXPORT_URL_TTL_HOURS	без лимита
archive_cleanup	02:30 UTC ежедневно	node-cron / Vercel Cron	Удаление архивов старше ARCHIVE_RETENTION_DAYS	100 файлов/з апуск

Таблица 13.1Б. Периодические cron-задачи. Минимальный интервал Vercel Cron — 1 минута.

Все задачи идемпотентны: повторный запуск безопасен (SELECT с фильтром by-time, UPDATE с WHERE status=pending). В Docker-окружении cron реализуется через node-cron внутри Worker-процесса; в Vercel — через Vercel Cron, вызывающий endpoints /api/cron/retention, /api/cron/grace-delete, /api/cron/backup (авторизация CRON_SECRET). При использовании node-cron в standalone Worker — restart: always обеспечивает восстановление после сбоя.

9.8 Резервное копирование

Трёхуровневая стратегия backup

Резервное копирование реализовано через отдельную модель BackupJob (см. §2.3) для полных дампов БД. Не переиспользуется ExportJob — он предназначен для одной сессии и имеет NOT NULL sessionId.

Резервное копирование реализовано на трёх уровнях для defence-in-depth:

Уровень	Метод	Авторизация	Частота	Retention	Назначение
1. Managed	Turso automated snapshots	Turso platform (managed)	Ежечасно + перед миграцией	7 дней (free) / 30 дней (Pro)	Защита от сбоя кластера, point-in-time recovery

Уровень	Метод	Авторизация	Частота	Retention	Назначение
2. Logical dump	POST /api/admin/backup → создаёт BackupJob (не ExportJob)	Bearer ADMIN_TOKEN	Ежедневно 03:00 UTC (cron в Worker)	90 дней (rolling), EXPORT_STORAGE_DIR	Защита от логического повреждения (accidental DROP, баг миграции)
3. On-demand export	POST /api/sessions/[id]/export (см. §4.11)	Bearer API_KEY или cookie	По запросу пользователя	24 ч TTL	Бэкап отдельных сессий перед удалением

Таблица 13.2. Стратегия резервного копирования (BackupJob, ADMIN_TOKEN, type=full).

Процедура восстановления (RTO/RPO): RPO (Recovery Point Objective) = 1 час (от Turso snapshots); RTO (Recovery Time Objective) = 30 минут (restore snapshot + restart контейнеров). Для logical dump (уровень 2) — RPO = 24 часа, RTO = 2 часа (требует импорта JSON обратно в БД через scripts/restore-backup.ts). Процедура восстановления документирована в runbook docs/runbooks/disaster-recovery.md и тестируется раз в квартал (chaos engineering: simulate DB failure → restore → smoke tests).

Backup-verification: ежедневный cron-задача в Worker проверяет целостность последнего BackupJob: открывает JSON, валидирует наличие всех таблиц, считает количество записей, сравнивает с COUNT(*) в БД, проверяет checksum (sha256). При расхождении > 5% — alert. Это обнаруживает «молчаливую коррупцию» бэкапов (silent corruption), когда файлы создаются, но содержимое повреждено. При успехе — BackupJob.verifiedAt = now(), status = "verified". Метрика: backup_verification_total{status=ok|fail}.

Правка 2.5 — обработка ошибок логического дампа

При ошибке создания BackupJob (уровень 2): (а) alert BackupFailed, уровень error; (б) метрика backup_logical_dump_total{status="fail"} увеличивается; (в) повтор через 1 час, максимум 3 попытки (attempts++, exponential backoff); (г) после 3 неудач — alert BackupRepeatedlyFailed, уровень critical, уведомление оператору; (д) запись в AuditLog: action="backup.failed", actorType="backup-cron", metadata={attempts, lastError}. При успехе после retry — AuditLog action="backup.succeeded_after_retry".

Expand-contract при миграции БД

Перед каждой миграцией (expand или contract phase) Worker создаёт дополнительный snapshot через Turso API (не дожидаясь ежечасного). Это даёт точку отката на момент до миграции. При неудаче миграции — восстановление snapshot, но только если миграция не успела применить деструктивные изменения (что запрещено expand-contract — см. §9.3). В случае rollback кода при успешной expand-миграции БД остаётся на новой схеме (backward-compatible).

10. Зависимости

Ключевые пакеты: `@upstash/redis` (Redis rate-limit), `p-limit` (controlled concurrency, уже был, теперь используется и в ingest DB-write), `nanoid` (генерация `clientId` при необходимости). Также: `xmlbuilder2` (генерация GPX/KML), `node-cron` (расписание cron-задач в standalone Worker).

Пакет	Версия	Назначение	Новое?
next	16.x	Фреймворк (App Router, API Routes, SSR)	
react	19.x	UI-библиотека	
react-leaflet	5.x	Компоненты карты Leaflet для React	
leaflet	1.9.x	Библиотека интерактивных карт	
@prisma/client	6.x	Клиент ORM для доступа к БД	
prisma	6.x (dev)	CLI для миграций и генерации	
@libsql/client	0.x	Драйвер Turso/LibSQL	
tailwindcss	4.x	Utility-first CSS-фреймворк	
zod	3.x	Валидация входных данных	
pino	9.x	Структурированное логирование	
p-limit	5.x	Controlled concurrency (ingest + traffic)	исп. в новом месте
prom-client	15.x	Prometheus-метрики	
@upstash/redis	1.x	Redis sliding window rate-limit	✓
nanoid	5.x	генерация clientId (fallback)	✓
xmlbuilder2	3.x	генерация GPX 1.1 / KML 2.2 XML	✓
node-cron	3.x	расписание cron-задач в Worker	✓
opossum	8.x	circuit breaker для 2ГИС API	✓
vitest	3.x	Unit и integration тесты	
@playwright/test	1.x	E2E-тесты	
msw	2.x	Моки HTTP-запросов в тестах	
k6	0.50+	Нагрузочные тесты (nightly)	

Таблица 15. Зависимости.

11. Переменные окружения

Переменные: REDIS_URL, MAX_PAYLOAD_BYTES, INGEST_CONCURRENCY, WORKER_POLL_INTERVAL_MS, WORKER_ID, RATE_LIMIT_BACKEND. Retention/export переменные: RETENTION_DAYS, GRACE_PERIOD_DAYS, EXPORT_ASYNC_THRESHOLD, EXPORT_STORAGE_DIR и др. Ни одна переменная не использует префикс NEXT_PUBLIC_. Все токены — server-side only. В Docker — передаются через secret-файлы или encrypted env. В Vercel — через Environment Variables (Production/Preview/Development).

Переменная	Описание	Пример	Server-only
DATABASE_URL	Строка подключения к БД	file:./dev.db или libsql://...	Да
API_KEY	Токен для server-side API	случайная строка 32+ символов	Да
INGEST_TOKEN	Токен для ingest	случайная строка 32+ символов	Да
CRON_SECRET	Секрет для Worker API и cron	случайная строка 32+ символов	Да
DGIS_ROUTING_KEY	API-ключ 2ГИС	8b34056d-...	Да
ALLOWED_ORIGIN	Разрешённый CORS origin	https://telemetry.example.com	Да
LOG_LEVEL	Уровень логирования	info (prod) / debug (dev)	Да
REDIS_URL	Redis для rate-limit	rediss://:pwd@host:port	Да
RATE_LIMIT_BACKEND	redis memory	redis (prod) / memory (dev)	Да
MAX_PAYLOAD_BYTES	лимит ingest payload	262144 (256 КБ)	Да
INGEST_CONCURRENCY	p-limit для DB-write	1	Да
WORKER_POLL_INTERVAL_MS	интервал опроса очереди	5000	Да
WORKER_ID	идентификатор Worker-инстанса	uuid (auto-gen)	Да
RATE_LIMIT_MAX_INGEST	Лимит ingest/мин	10	Да
RATE_LIMIT_MAX_DEFAULT	Лимит default/мин	60	Да
WORKER_BATCH_SIZE	задач за один poll	10	Да
WORKER_MAX_CONCURRENCY	p-limit внутри Worker	5	Да
RETENTION_DAYS	срок хранения сессий	3650 (10 лет)	Да
RETENTION_ARCHIVE_ENABLED	авто-архивация перед удалением	true	Да

Переменная	Описание	Пример	Server-only
RETENTION_CRON_UTC	расписание retention-cron	"30 0 * * *"	Да
ARCHIVE_RETENTION_DAYS	срок хранения архивов	3650 (10 лет)	Да
GRACE_PERIOD_DAYS	grace period для soft-delete	30	Да
AUDIT_RETENTION_DAYS	срок хранения AuditLog	3650 (10 лет)	Да
EXPORT_ASYNC_THRESHOLD	порог аsync-экспорта (точек)	5000	Да
EXPORT_STORAGE_DIR	директория файлов экспорта	/data/exports	Да
EXPORT_URL_TTL_HOURS	TTL ссылок на файлы	24	Да
EXPORT_MAX_FILE_BYTES	макс. размер одного файла	104857600 (100 МБ)	Да
EXPORT_CLEANUP_CRON_UTC	расписание очистки файлов	"0 * * * *" (каждый час)	Да
VERCEL_PLAN	free pro (CI-проверка нагрузки)	pro (для 100 сессий/мин)	Да
TARGET_LOAD_RPM	целевая нагрузка (сессий/мин)	100	Да
CIRCUIT_BREAKER_THRESHOLD	отказов 2ГИС до размыкания	5	Да
CIRCUIT_BREAKER_TIMEOUT_SEC	таймаут half-open	30	Да
BACKUP_LOGICAL_CRON_UTC	расписание logical dump	"0 3 * * *" (03:00 UTC)	Да
BACKUP_RETENTION_DAYS	retention logical dump	90	Да
BACKUP_VERIFICATION_ENABLED	проверка целостности backup	true	Да
ADMIN_TOKEN	токен для административных операций (backup, restore, audit-log)	случайная строка 32+ символов	Да
BACKUP_MAX_ATTEMPTS	максимум попыток BackupJob	3	Да
BACKUP_RETRY_INTERVAL_HOURS	интервал retry при fail	1	Да

Таблица 16. Переменные окружения.

Статус документа. Спецификация v2.6 описывает production-grade архитектуру сервиса «Телеметрия поездов» и является обязательной к исполнению для всех коммитов после 29.08.2026. Оценка готовности: **8.0/10**. Система обладает полным набором production-гарантий: STRIDE threat model,

snap-to-grid кэш, верифицируемый Worker-захват через RETURNING id, BackupJob с type=full, ADMIN_TOKEN для всех административных операций, машиночитаемые правила AlertManager, согласованные Prisma @relation-связи, однопользовательская модель авторизации. **Незавершённые работы:** реальные k6-замеры (таблица 12 содержит симулированные данные) и фактическое покрытие тестами (таблица 11.1 — симуляция) требуют проведения в CI до release в production. После подтверждения симулированных данных реальными замерами возможно повышение оценки до 9.0.